



Runtime composition in dynamic system of systems: A systematic review of challenges, solutions, tools, and evaluation methods

Muhammad Ashfaq ^{a,*}, Ahmed R. Sadik ^b, Teerath Das ^a, Muhammad Waseem ^c,
Niko Mäkitalo ^a, Tommi Mikkonen ^a

^a Faculty of IT, University of Jyväskylä, 40014, Jyväskylä, Finland

^b Honda Research Institute Europe, 63073, Offenbach, Germany

^c Faculty of Information Technology and Communications, Tampere University, 33100, Tampere, Finland

ARTICLE INFO

Editor: Dr. Jacopo Soldani

Keywords:

System of systems
Runtime composition
Dynamic reconfiguration
Self-adaptation
Self-integration
Interoperability

ABSTRACT

Context: Modern Systems of Systems (SoSs) increasingly operate in dynamic environments (e.g., smart cities, autonomous vehicles) where *runtime composition*—the on-the-fly discovery, integration, and coordination of constituent systems (CSs)—is crucial for adaptability. Despite growing interest, the literature lacks a cohesive synthesis of runtime composition in dynamic SoSs.

Objective: This study synthesizes research on runtime composition in dynamic SoSs and identifies core challenges, solution strategies, supporting tools, and evaluation methods.

Methods: We conducted a Systematic Literature Review (SLR), screening 1774 studies published between 2019 and 2024 and selecting 80 primary studies for thematic analysis (TA).

Results: Challenges fall into four categories: modeling and analysis, resilient operations, system orchestration, and heterogeneity of CSs. Solutions span seven areas: co-simulation and digital twins, semantic ontologies, integration frameworks, adaptive architectures, middleware, formal methods, and AI-driven resilience. Service-oriented frameworks for composition and integration dominate tooling, while simulation platforms support evaluation. Interoperability across tools, limited cross-toolchain workflows, and the absence of standardized benchmarks remain key gaps. Evaluation approaches include simulation-based, implementation-driven, and human-centered studies, which have been applied in domains such as smart cities, healthcare, defense, and industrial automation.

Conclusions: The synthesis reveals tensions, including autonomy versus coordination, the modeling-reality gap, and socio-technical integration. It calls for standardized evaluation metrics, scalable decentralized architectures, and cross-domain frameworks. The analysis aims to guide researchers and practitioners in developing and implementing dynamically composable SoSs.

1. Introduction

In today's interconnected world, modern systems increasingly operate in dynamic and unpredictable environments. In this context, the concept of a System of Systems (SoS) has emerged as an innovative paradigm for designing complex software ecosystems that comprise independently developed and managed constituent systems (CSs).¹ These

CSs must interoperate and collaborate to achieve overarching goals beyond any individual system's scope. Examples of SoSs include patient monitoring systems and smart cities, each integrating and coordinating diverse CSs to achieve overarching objectives.

Due to volatile environments, modern SoSs require continuous adaptation and real-time reconfiguration. Consequently, *runtime composition*—the ability to dynamically discover, integrate, and coordinate CSs

* Corresponding author.

E-mail addresses: muhammad.m.ashfaq@jyu.fi (M. Ashfaq), ahmed.sadik@honda-ri.de (A.R. Sadik), teerath.t.das@jyu.fi (T. Das), muhammad.waseem@tuni.fi (M. Waseem), niko.k.makitalo@jyu.fi (N. Mäkitalo), tommi.j.mikkonen@jyu.fi (T. Mikkonen).

¹ SoS refers to a single System of Systems and SoSs to multiple Systems of Systems; CS refers to a single Constituent System and CSs to multiple Constituent Systems.

at runtime—has become a crucial mechanism for responding to partially known or unforeseen environmental conditions.

Runtime composition with unknown CSs presents a fundamental challenge in System of Systems Engineering (SoSE) (INCOSE, 2014; IEEE, 2016). While SoSs can reconfigure based on pre-stored environmental inputs with known CSs, the challenge of discovering and integrating unforeseen CSs that are independent, heterogeneous, and distributed (Section 2) during operation remains largely unsolved (Fang, 2022).

This capability is particularly relevant for industrial applications such as urban mobility and energy SoSs. Projects such as the Honda eVTOL initiative (eVTOL, 2025), connected vehicle fleets (Sadik et al., 2025; Ashfaq et al., 2024), large-scale trials of bidirectional electric vehicle charging (FCR, 2025), and vehicle-to-building energy management (Stadie et al., 2021) illustrate the potential of runtime composition. If achieved, it can enhance interoperability, vehicle-to-infrastructure coordination, and fleet management. Beyond the mobility sector, open-source frameworks like CityLearn (Nweye et al., 2025) exemplify the broader industrial and societal relevance of dynamic SoS solutions. Such initiatives highlight the practical challenges and emerging solutions in runtime composition, underscoring the need for a systematic review to consolidate current knowledge and guide future research.

Despite growing interest in runtime composition in dynamic SoSs (Section 2.4), the literature lacks a cohesive synthesis. This Systematic Literature Review (SLR) addresses this gap by: (1) identifying key challenges to runtime composition, (2) analyzing proposed solutions and frameworks, (3) identifying supporting tools, and (4) evaluation methods. This study makes the following key contributions through a systematic and in-depth synthesis of the literature on runtime composition in dynamic SoSs:

- *A unified taxonomy of runtime composition challenges*, categorized into four dimensions: modeling and analysis, resilient operations, system orchestration, and heterogeneity.
- *A comprehensive mapping of solution strategies*, encompassing digital twins, semantic ontologies, integration frameworks, formal methods, and artificial intelligence (AI) driven resilience mechanisms.
- *An analysis of supporting tools and platforms*, highlighting both well-established environments and current limitations in tool support.
- *A structured assessment of evaluation methods*, including commonly used methods and application domains.
- *The identification of critical research gaps*, including the need for standardized benchmarking, unified modeling of emergent behavior, scalable decentralized control, cross-domain interoperability, and vendor-agnostic middleware.

The remainder of this study is structured as follows. Section 2 presents the background on dynamic SoSs and runtime composition. Section 3 describes the SLR methodology, detailing the planning, execution, and reporting phases. Section 4 reports the findings and synthesized taxonomies derived from the selected studies. Section 5 interprets these findings, discussing their implications for research and practice. Section 6 identifies limitations of the SLR. Section 7 situates our SLR within the broader literature. Finally, Section 8 concludes the paper and outlines directions for future research.

2. Background

This section introduces SoS, including its characteristics, types, and architectures, providing context for our SLR and motivating the problem statement. Readers interested in a more comprehensive treatment are referred to (Jamshidi, 2008) and (Nielsen et al., 2015).

An SoS is an ensemble of operationally and managerially independent systems that achieve objectives beyond any individual CS's capabilities. An example of an SoS is a patient monitoring system that integrates CSs, such as information systems, measurement components, and diagnostic analysis tools (Benabidallah et al., 2020). While each CS has its

well-defined mission, its ensemble serves the broader purpose of maintaining patient stability, a goal beyond the capacity of any single CS. Similarly, smart cities represent another example of SoS, where diverse systems must be prepared to join and contribute, even temporarily (Teixeira et al., 2020). SoSs can be found in various domains, including smart grids, supply chains, intelligent transportation systems, emergency response networks, and healthcare infrastructures.

2.1. SoS Characteristics

SoSs are distinguished from monolithic systems based on several characteristics. These properties fall into two categories: CS-level characteristics and SoS-level characteristics.

CS-level characteristics include geographic distribution, as they can be physically dispersed, and two forms of independence: operational and managerial (O/M independence). Operational independence means that each CS can function autonomously and would continue to operate even if removed from the SoS. Managerial independence implies that each CS is governed separately, with its goals, resources, and development plans. This O/M independence enables CSs to join, leave, and adapt within an SoS without disrupting their internal operations or management structures (Song et al., 2022). Closely related to independence is *autonomy* (Mittal et al., 2020), where CSs maintain the ability to achieve goals while operating independently of external control, extending beyond mere automation to include *behavioral autonomy* in both static contexts and dynamic environments. In this way, behavioral autonomy ensures adaptability at the SoS level, while *structural autonomy* enables infrastructure to incorporate CSs dynamically.

At the SoS level, two characteristics particularly distinguish these systems: *evolutionary development* and *emergent behavior*. An SoS is rarely complete but instead undergoes dynamic evolution (Fang, 2022) in which new CSs are added, current CSs are replaced or removed, and the interdependency network can switch to a more efficient structure over time. The interactions among CSs lead to SoS-level behaviors, both intended and unintended, not present in or predictable from individual CSs. For example, a traffic jam is an emergent misbehavior (unintended emergent behavior) as no single vehicle is designed to cause congestion, yet it arises from their collective interactions. In contrast, vehicle platooning is an intended emergent behavior, with cars coordinating to form convoys and optimize traffic flow (Oquendo, 2019a).

Additional characteristics proposed in the literature to further distinguish SoS from traditional systems include belonging, connectivity, diversity, networks, heterogeneity, and trans-domain operation. However, several characteristics overlap (e.g., connectivity and network topology) and can be merged to streamline the definitional framework.

2.2. SoS types

Based on their coordination mode, (ISO/IEC/IEEE, 2019) defines four SoS types: directed, acknowledged, collaborative, and virtual. *Directed SoS* features a central authority that creates and manages the SoS for a specific purpose, with CSs subordinated to the SoS goals. A military operation center controlling various units exemplifies this type. The *acknowledged SoS* has recognized objectives and a designated manager, but CSs remain independently owned and managed; changes occur by cooperative agreement. A federated consortium of companies sharing data under central governance illustrates this type. *Collaborative SoS* lacks a single authority, but CSs voluntarily cooperate toward agreed goals, collectively deciding how to share services. Independent firms jointly forming a supply chain represent this model. A *virtual SoS* has neither central management nor predefined purpose; large-scale behaviors emerge from loosely coupled CSs. An example is an open network of devices on the internet that self-organize to provide a service.

Except in Directed SoS, CS must balance autonomy and belonging (Fang, 2022); designers and managers cannot unilaterally dictate

when a CS joins or leaves the SoS, since each CS's O/M independence governs its participation.

This classification represents a spectrum from most to least centrally directed, with the reverse order indicating the extent of CS coordination authority. The type of SoS fundamentally shapes how CSs are integrated to achieve overarching goals.

2.3. SoS architectures

The architecture of an SoS specifies its CSs, the interactions among them, and the mediators enabling those interactions (Silva et al., 2020). Architecturally, an SoS can be viewed from two perspectives: (1) a set of abstract-level decisions encoded as variables with allowed alternatives and (2) description models supported by architecture frameworks (Fang, 2022).

Four primary architectural styles have emerged to address the diverse needs of different SoS contexts. A *centralized architecture* features a single controller issuing commands to all CSs through top-down orchestration, simplifying coordination but introducing a single point of failure and limiting scalability. A *hierarchical architecture* distributes control across multiple layers where higher-level systems coordinate lower-level subsystems, improving flexibility while maintaining central oversight. *Heterarchical architectures* eliminate central control, with CSs cooperating through peer-to-peer interactions, offering fault tolerance at the cost of coordination complexity. The *holonic architecture* models each CS as a *holon*—a semi-autonomous unit functioning both as a self-contained system and as part of a larger hierarchy—enabling both bottom-up assembly and top-down decomposition of goals (Blair et al., 2015).

These architectural styles align with the SoS types (Section 2.2). Directed SoSs employ centralized architectures to maintain tight control over CSs. Acknowledged SoSs generally implement hierarchical architectures that balance central coordination with local autonomy. Collaborative SoSs benefit from heterarchical architectures where coordination emerges through consensus rather than top-down control. Virtual SoSs often employ heterarchical or holonic architectures, allowing system behavior to emerge organically without formal coordination structures.

2.4. The shift towards dynamic SoSs

Modern SoS environments require CSs to react jointly to runtime changes from other CSs and the external environment (Wudka et al., 2020). This dynamic context necessitates *runtime composition*—the ability to discover, integrate, and reassemble CSs on-the-fly despite heterogeneity in interfaces, behaviors, and objectives. Such capabilities allow CSs to join, leave, be replaced, or be modified within the operational SoS (Neto, 2017), enabling adaptation and evolutionary development (Forte et al., 2022).

Achieving robust runtime composition is challenging for several interrelated reasons. Most CSs have limited individual capabilities, necessitating integration to create required emergent behaviors (Wudka et al., 2020). These CSs are highly heterogeneous, originating from different providers, maintained by distinct entities (Lopes et al., 2016), and designed with specific missions rather than interoperability in mind (Nadira et al., 2020). They differ in protocols, data formats, workflows, and interfaces (Garcés et al., 2019). Furthermore, CSs often function as *black boxes* where only interfaces are known, while internal structures and behaviors remain inaccessible (Silva et al., 2020).

Compounding these difficulties is uncertainty about which CSs will participate and the operational environment (Oquendo, 2019c,b, 2020; Agarwal et al., 2023)—a concern emphasized by both INCOSE (INCOSE, 2014) and the European Commission's Cyber-Physical SoS (CPSoS) Research Agenda (IEEE, 2016).

Current engineering methodologies have limitations in adequately addressing these challenges. Existing approaches remain predominantly static, making evolution difficult once CSs become outdated (Zabasta et al., 2020). Most implementations rely on predefined adaptation

strategies, which become infeasible when attempting to anticipate all possible scenarios (Oquendo, 2019b; Lee et al., 2021; Oquendo, 2020). These predefined adaptation strategies degrade performance when discrepancies exist between design-time knowledge and runtime events (Lee et al., 2021). Furthermore, several solutions take a top-down engineering approach, unsuitable for future SoSs that require bottom-up, autonomous integration (Teixeira et al., 2020).

Achieving robust on-the-fly composition raises a spectrum of open questions ranging from translating between disparate data formats to guaranteeing safety when reconfiguring black-box components.

3. Methodology

Our methodology is guided by the well-established procedures of Kitchenham et al. (2015) and comprises three phases: (i) planning the review (Section 3.1), (ii) executing the review (Section 3.2), and (iii) reporting (Section 3.3), as shown in Fig. 1. To ensure transparency and replicability, we provide a replication package, including our search string, inclusion/exclusion steps, data extraction forms, quality assessments, and synthesized data Replication-Package (2025).

3.1. Planning phase

3.1.1. Research objective and SLR design

This research aims to synthesize the literature on runtime composition in dynamic SoSs by identifying core challenges, existing solutions, supporting tools, and evaluation methods. To achieve this, we conducted an SLR that consolidates current knowledge in the field.

While a Systematic Mapping Study (SMS) can also be used for surveying the literature (Petersen et al., 2008), such studies primarily aim to provide a broad classification of research topics and trends. In contrast, our focused and integrative objectives require the deeper analysis and interpretive synthesis characteristic of an SLR.

3.1.2. Research questions

Table 1 presents the four RQs and their underlying rationales, which structure and guide this SLR.

3.1.3. Search strategy

Our search strategy was conducted in two complementary phases to maximize coverage of relevant primary studies.

Phase 1: Automated database search. We identified six widely used digital libraries as our primary search sources: ACM Digital Library, IEEE Xplore, Elsevier Scopus,² SpringerLink, Wiley Online Library, and Web of Science. These databases were chosen for their comprehensive coverage of computing and engineering literature and frequent use in similar SLRs (Zhang et al., 2011). No manual venue-based search or expert contact was conducted, as the selected libraries already provide broad and overlapping coverage of relevant publication venues.

The search string was derived from our RQs and key concepts under investigation. To calibrate the string, we manually curated a baseline reference set of 13 high-quality studies, ensuring these studies represented the most relevant and influential work in the targeted research area. Keywords were then extracted from both the RQs and the baseline studies and combined using logical operators to form the initial search string.

The final search string applied across all six libraries was: ('system-of-systems' OR 'systems-of-systems' OR 'system of systems' OR 'systems of systems') AND ('runtime composition' OR 'dynamic reconfiguration' OR 'self-adaptation' OR 'self-integration' OR

² Elsevier Scopus offers broader coverage compared to Elsevier ScienceDirect (Galaway, 2024).

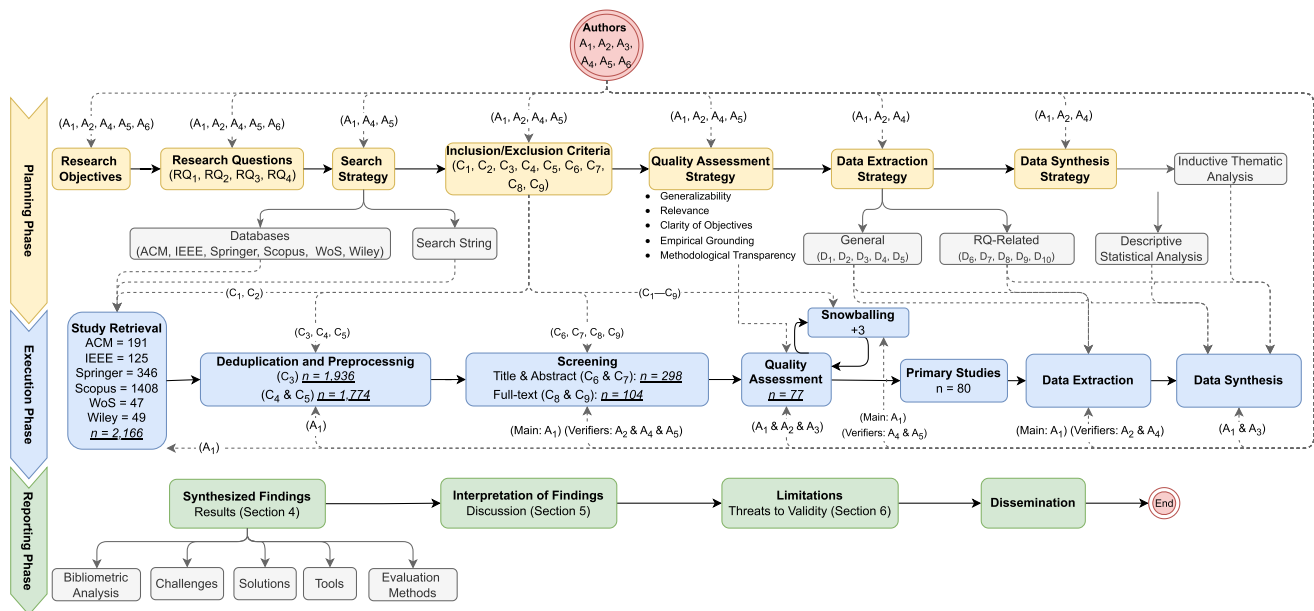


Fig. 1. Methodology adopted for the SLR, illustrating its planning, execution, and reporting phases.

Table 1
Research questions and their corresponding rationales.

#	Research Question	Rationale
RQ ₁	What are the primary challenges to achieving runtime composition in dynamic SoSs?	Identifying and characterizing the key challenges offers foundational insight into what hinders the realization of runtime composition in SoS contexts. This understanding is essential for framing the research landscape and guiding the development of targeted solutions.
RQ ₂	What solutions (approaches, conceptual frameworks, and architectural strategies) have been proposed to achieve runtime composition in dynamic SoSs?	A qualitative synthesis of proposed solutions can reveal underlying design philosophies, recurring assumptions, and emerging architectural patterns. This can guide future innovations and help practitioners align strategies with state-of-the-art thinking.
RQ ₃	What tools, platforms, or software frameworks have been used or proposed to support runtime composition in dynamic SoSs?	Supporting practical tools is essential for operationalizing research ideas. This RQ provides insight into the field's technological maturity and highlights the gap between conceptual approaches and usable, integrable toolchains in real-world settings.
RQ ₄	How are runtime composition approaches evaluated in the literature, and what application domains are considered?	This RQ explores the evaluation methods and contexts used to assess runtime composition solutions. Understanding the rigor and diversity of evaluation methods, along with the domains in which solutions are applied, helps gauge current research's maturity and practical relevance.

Phase 2: Snowballing. We applied backward and forward snowballing following Wohlin’s guidelines (Wohlin, 2014) to complement the automated search. Consistent with best practice, the *start set* for snowballing was defined as the set of primary studies retained after screening and quality assessment, ensuring that iterations would be based on a reliable corpus of relevant studies. All candidate studies identified through snowballing were subjected to the same inclusion, exclusion, and quality criteria as in Phase 1.

3.1.4. Inclusion and exclusion criteria

We defined the inclusion and exclusion criteria based on insights from a pilot run of the search string to refine the search results. Table 2 presents each criterion along with the execution phase in which it was applied.

3.1.5. Quality assessment strategy

We designed a structured quality assessment rubric to ensure methodological rigor and consistency in study inclusion. Each study was evaluated against five predefined quality criteria:

1. *Relevance*: The study addresses runtime composition or related challenges in the SoS context.
2. *Clarity of Objectives*: The research problem, aim, or hypothesis is clearly stated.
3. *Methodological Transparency*: Sufficient methodological detail is provided.
4. *Empirical Grounding*: The study presents empirical evaluation (e.g., experiments, case studies).
5. *Generalizability*: The findings are applicable beyond a single domain or system.

Each criterion was scored as 0, 0.5, or 1. The total score per study ranges from 0 to 5. Studies scoring > 2 were considered to meet the quality standard and were included in the SLR.

3.1.6. Data extraction strategy

To address the RQs, we defined a structured schema of data items (Table 3). These items are grouped into two categories: (1) general study characteristics (D_1 – D_5) capturing metadata and demographics, and (2) thematic elements (D_6 – D_{10}) supporting analysis aligned with the RQs.

```
'interoperability') AND (framework* OR method* OR tool*
OR architecture)
```

The replication package includes the derived keywords from the RQs, the baseline reference set, and the keywords extracted from the baseline reference set.

Table 2

Inclusion and exclusion criteria, along with the review execution phase, where each criterion was applied.

ID	Inclusion Criteria	Exclusion Criteria	Execution Phase
C ₁	Studies must have been published within the last five years (January 1, 2019–September 22, 2024).	Studies published outside this timeframe.	Study Retrieval (Section 3.2.1)
C ₂	Studies must be written in English.	Studies in other languages (e.g., Chinese, Spanish, French).	Study Retrieval (Section 3.2.1)
C ₃	Studies must be unique (most recent version retained, e.g., journal extensions of conference papers).	Duplicate versions of the same study.	Deduplication and Preprocessing (Section 3.2.2)
C ₄	Studies must be peer-reviewed published articles, conference papers, or book chapters.	Non-peer-reviewed or grey literature (e.g., blog posts), reference works, books, theses, and review articles*.	Deduplication and Preprocessing (Section 3.2.2)
C ₅	Studies must be published as individual research articles or book chapters.	Full journals, conference proceedings volumes, editorials, prefaces, and retracted articles.	Deduplication and Preprocessing (Section 3.2.2)
C ₆	Studies must originate from the Computer Science or closely related engineering domains, focusing on computational, architectural, or systems aspects.	Studies outside the relevant domains (e.g., Business & Management).	Title & Abstract Screening (Section 3.2.3)
C ₇	Studies must explicitly mention the SoS context in the title or abstract.	Studies that lack SoS context in both the title and abstract.	Title & Abstract Screening (Section 3.2.3)
C ₈	Studies must describe a solution or approach relevant to the research subject and provide sufficient methodological detail for assessment.	Studies that lack methodological detail or are non-substantive (e.g., vision papers, fast abstracts, opinion articles).	Full-text Screening (Section 3.2.3)
C ₉	Studies must explicitly address runtime composition in dynamic SoSs or related challenges, solutions, tools, or evaluation methods.	Studies unrelated to runtime composition in dynamic SoS.	Full-text Screening (Section 3.2.3)

*Review articles are not considered as primary studies; however, relevant review articles are included as related works (Section 7).

Table 3

Data items to be extracted from the selected studies and their corresponding RQs.

Code	Data Item	Description	RQ
D ₁	Study ID	Unique identifier for the study to mention in the manuscript (e.g., S01, S02, ...).	Overview
D ₂	Author(s)	List of authors to find the prominent authors.	Overview
D ₃	Publication Year	For publication timeline and trend analysis.	Overview
D ₄	Study Title	Full title of the study.	Overview
D ₅	Publication Type	Journal article, conference paper, book chapter, etc.	Overview
D ₆	Challenges Identified	Specific challenges mentioned (e.g., heterogeneity, interoperability, self-integration).	RQ ₁
D ₇	Proposed Solution	The proposed method, process, framework, or architecture.	RQ ₂
D ₈	Tools & Technologies	Tools, technologies, or software used to implement the solution.	RQ ₃
D ₉	Evaluation Methods	Evaluation type (e.g., case study, simulation, empirical study).	RQ ₄
D ₁₀	Application Domain	Application domains addressed in the selected studies.	RQ ₄

3.1.7. Data synthesis strategy

We applied a hybrid data synthesis approach reflecting the nature of the extracted data. Quantitative items (D1–D5) were analyzed using descriptive statistics to identify publication trends and metadata patterns. Qualitative items (D6–D10) were synthesized using reflexive thematic analysis (TA), following Braun and Clarke’s six-phase method (Braun and Clarke, 2006, 2019). This approach is well-suited to synthesizing heterogeneous qualitative data common in software engineering reviews (Cruzes and Dyba, 2011).

3.1.8. Team roles and responsibilities

The review team’s roles and responsibilities were distributed across different phases of the SLR to ensure methodological rigor and reduce potential bias. For clarity, the authors are referred to as A₁ through A₆ throughout this section. A₁ served as the review lead throughout the process, consistent with Kitchenham’s recommendation for PhD-led reviews (Kitchenham et al., 2015).

The *research objective* (Section 3.1.1) and RQs (Section 3.1.2) were collaboratively formulated by A₁, A₂, A₄, A₅, and A₆, drawing on their combined expertise in software and systems engineering.

The *search strategy* (Section 3.1.3) was jointly designed by A₁, A₄, and A₅. A₁ and A₄ conducted pilot searches and refined the search string. Subsequently, A₁ executed the final search, managed the digital library queries, and consolidated the initial dataset (Section 3.2.1).

A₁, A₂, A₄, and A₅ jointly developed the *inclusion and exclusion criteria* (Section 3.1.4). *Screening* (Section 3.2.3), including *title and abstract screening* and *full-text screening*, was led by A₁, who applied these criteria. A₂, A₄, and A₅ independently verified a subset of screening decisions. Discrepancies were resolved through discussion and consensus.

A₁, A₂, A₄, and A₅ jointly developed the *quality assessment strategy* (Section 3.1.5), while A₁, A₂, and A₃ independently assessed the quality of each study in full.

The *snowballing process* (Section 3.2.5) was primarily conducted by A₁. A₄ and A₅ independently verified the findings and supported the validation of included studies.

The *data extraction* (Section 3.1.6) and *synthesis strategy* (Section 3.1.7) were co-developed by A₁, A₂, and A₄. Following this, A₁ performed the initial *data extraction* (Section 3.2.6). A₂ and A₄ independently reviewed a subset of data items (D₆–D₁₀) that required interpretive judgment to confirm coding consistency and minimize the risk of misclassification. The data synthesis phase (Section 3.2.6) was carried out through close collaboration between A₁ and A₃, who jointly performed inductive coding and developed the thematic structure.

For verification, independent checks were performed on randomly sampled studies. Discrepancies were resolved in joint meetings, and whenever these discussions led to refinements in the criteria or assessment guidelines, the updates were consistently applied across all studies.

3.2. Execution phase

3.2.1. Study retrieval

The search string was adapted for each digital library to account for syntax differences and platform-specific constraints, as presented in Table 4.

The search scope was set to *all* available fields, targeting titles, abstracts, keywords, metadata, and other relevant fields to maximize coverage. This comprehensive approach increased the number of retrieved studies and enhanced the likelihood of capturing relevant literature across different indexing strategies.

Table 4

Data sources and their respective search strings.

Database	Search String
ACM	[[All: 'system-of-systems']] OR [All: 'systems-of-systems']] OR [All: 'system of systems']] OR [All: 'systems of systems']] AND [[All: 'runtime composition']] OR [All: 'dynamic reconfiguration']] OR [All: 'self-adaptation']] OR [All: 'self-integration']] OR [All: 'interoperability']] AND [[All: framework*]] OR [All: method*]] OR [All: tool*]] OR [All: architecture]] AND [E-Publication Date: Past 5 years]
IEEE	('system-of-systems' OR 'systems-of-systems' OR 'system of systems' OR 'systems of systems') AND ('runtime composition' OR 'dynamic reconfiguration' OR 'self-adaptation' OR 'self-integration' OR 'interoperability') AND (framework* OR method* OR tool* OR architecture)
Wiley	'<ddq>'system-of-systems'</ddq> OR <ddq>'systems-of-systems'</ddq> OR <ddq>'system of systems'</ddq> OR <ddq>'systems of systems'</ddq>' anywhere AND '<ddq>'runtime composition'</ddq> OR <ddq>'dynamic reconfiguration'</ddq> OR <ddq>'self-adaptation'</ddq> OR <ddq>'self-integration'</ddq> OR <ddq>'interoperability'</ddq>' anywhere AND 'framework* OR method* OR tool* OR architecture' anywhere
Springer	('system-of-systems' OR 'systems-of-systems' OR 'system of systems' OR 'systems of systems') AND ('runtime composition' OR 'dynamic reconfiguration' OR 'self-adaptation' OR 'self-integration' OR 'interoperability') AND (framework* OR method* OR tool* OR architecture); within English, (Computer Science Engineering), 2019 - 2025
Scopus	ALL('system-of-systems' OR 'systems-of-systems' OR 'system of systems' OR 'systems of systems') AND ALL('runtime composition' OR 'dynamic reconfiguration' OR 'self-adaptation' OR 'self-integration' OR 'interoperability') AND ALL(framework* OR method* OR tool* OR architecture) AND PUBYEAR > 2018 AND PUBYEAR < 2026 AND (LIMIT-TO (SUBJAREA , 'COMP') OR LIMIT-TO (SUBJAREA , 'ENGI')) AND (LIMIT-TO (DOCTYPE , 'cp') OR LIMIT-TO (DOCTYPE , 'ar')) OR LIMIT-TO (DOCTYPE , 'ch')) AND (LIMIT-TO (LANGUAGE , 'English'))
WoS	ALL=('System of Systems' OR 'systems of systems' OR 'system-of-systems' OR 'systems-of-systems') AND ALL=('runtime composition' OR 'dynamic reconfiguration' OR 'self-adaptation' OR 'self-integration' OR 'interoperability') AND ALL=(framework* OR method* OR tool* OR architecture)

We applied inclusion/exclusion criteria C_1 and C_2 at this phase (Table 2).

Executing the adapted search strings across all selected databases yielded 2166 studies. The distribution of results per database is detailed in Fig. 1.

3.2.2. Deduplication and preprocessing

We observed duplicate studies during the search process, mainly due to WoS and Scopus indexing overlapping content from other databases. Applying inclusion/exclusion criterion C_3 (Table 2) reduced the dataset to 1936 studies.

Abstracts were required for the subsequent screening phase (Section 3.2.3). For studies missing abstracts, these were manually retrieved through online searches.

In addition, inclusion/exclusion criteria C_4 and C_5 were applied at this phase (Table 2), further refining the dataset to 1774 candidate studies.

3.2.3. Screening

Title & Abstract Screening: During this step, we reviewed titles and abstracts of the candidate studies, applying inclusion/exclusion criteria C_6 and C_7 (Table 2) to ensure domain relevance and SoS context. This screening reduced the dataset to 298 studies.

Full-text Screening: We conducted a thorough full-text review of the remaining studies, applying inclusion/exclusion criteria C_8 and C_9 (Table 2). For studies without accessible full texts, we contacted the authors via ResearchGate and excluded the studies if they were unavailable. This phase refined the dataset to 104 studies.

3.2.4. Quality assessment

Using the predefined quality assessment rubric (Section 3.1.5), each study was scored on a scale from 0 to 5, yielding a final set of 77 studies.

3.2.5. Snowballing

Backward snowballing produced 1832 studies, which were reduced to 1737 unique studies after deduplication. Forward snowballing yielded 484 studies, corresponding to 440 unique studies. Together, the two directions produced 2177 unique candidate studies.

After applying inclusion, exclusion, and quality criteria, two studies (S78 and S80) were added to the primary studies. In addition, four backward-snowballing studies (B54, B704, B873, B924) consistently cited a key work (S79) not retrieved by the automated search. Given its repeated citation and centrality, S79 was evaluated against our criteria and included.

In this way, snowballing added three further primary studies (all via backward snowballing), while forward snowballing yielded none. Snowballing brought the total to 80 primary studies. The final set, along with cumulative quality scores and identifiers (e.g., S01, S02, ...), is provided in Appendix A. The complete list of studies retrieved through snowballing is provided in the replication package.

3.2.6. Data extraction and synthesis

Following the planned extraction and synthesis strategies (Sections 3.1.6 and 3.1.7), descriptive statistics summarized data items D_1 – D_5 . For D_6 – D_{10} , reflexive TA was conducted, involving open coding, iterative theme refinement, and interpretive abstraction. We emphasized interpretive rather than objective theme identification, consistent with reflexive TA principles.

3.3. Reporting phase

This phase encompasses the reporting, interpretation, and communication of findings. These aspects are addressed in detail in the following sections:

- The reporting of results, including synthesized findings, is presented in Section 4.
- The interpretation of findings is discussed in Section 5.
- Study limitations are outlined in Section 6.

4. Results

We reported the results in dedicated sections aligned with our RQs (Table 1). Section 4.1 covers the demographic details of the selected studies. Section 4.2 and Section 4.3 detail the challenges (RQ₁) and proposed solutions (RQ₂), respectively. Section 4.4 describes the tools (RQ₃), and Section 4.5 outlines the evaluation methods (RQ₄).

4.1. Bibliometric analysis

4.1.1. Venue distribution

Fig. 2 visualizes the percentage distribution of publication types of the selected studies. Most studies appeared in conference or workshop proceedings (47), followed by journal articles (27), and a smaller number as book chapters (6).

Table 5 lists the top publication venues for the selected studies. The IEEE Systems Journal and SeSoS lead with four studies, followed by SESoS with three. Other notable venues include JSS, SysCon, INCOSE Journal, MILCOM, and CCIS, each contributing two studies, while the remaining venues contributed one study each. Full venue names and additional details are provided in the table.

Table 5
Most frequent individual publication venues in the selected studies.

Rank	Full Name	Studies
1	IEEE Systems Journal International Conference on System of Systems Engineering (SoSE)	4
2	IEEE/ACM International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems (SESoS)	3
3	Journal of Systems and Software (JSS) The Journal of International Council on Systems Engineering (Wiley INCOSE) IEEE International Systems Conference (SysCon) Communications in Computer and Information Science (CCIS) IEEE Military Communications Conference (MILCOM) IEEE/IFIP Network Operations and Management Symposium (NOMS) Industrial Electronics Conference (IECON) Reliability Engineering and System Safety	2

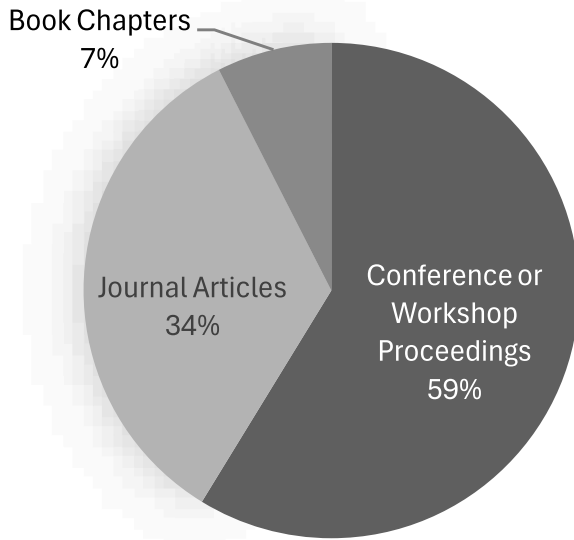


Fig. 2. Distribution of publication types for the selected studies.

4.1.2. Publication trends

Fig. 3a illustrates the publication timeline of the selected studies. Despite minor fluctuations, the overall trend remains consistent, with an average of slightly more than 10 studies published annually. Notably, 2019 and 2021 show peaks with 18 publications each, while the other years align closely with the average.

An analysis of the cumulative publications (**Fig. 3b**) reveals a strong linear growth pattern, supported by a high determination coefficient ($R^2 = 0.9926$). This growth shows that the research subject has been experiencing steady growth and increasing scholarly engagement, demonstrating the accelerating development of the research area.

4.2. Challenges (RQ₁)

Table 6 presents our taxonomy of challenges, organized in four themes.

4.2.1. Modeling and analysis

This theme reveals the limitations in representing and analyzing complex SoS behaviors. The *Unified System Modeling* subtheme exposes critical gaps in holistic representation, particularly in integrating quantitative resource allocation with qualitative features (S67) and bridging micro-macro system perspectives (S62). Current approaches struggle with multi-domain heterogeneity (S74) and emergent behavior prediction (S78), while the absence of formal computation models in control software (S07) compounds these issues.

The *Multi-Domain Simulation* subtheme highlights interoperability barriers, with studies demonstrating the need for standardized methods (S01) and federated simulations incorporating hardware/human interactions (S03, S47). Researchers emphasize requirements for dynamic property support in metamodels (S03) and architecture simulation across requirement variants (S57).

In *Verification & Validation*, key challenges include exponential cost growth during frequent reconfigurations (S33, S41) and accuracy limitations from poor knowledge sharing between CSs (S41). Physical validation environments remain prohibitively expensive (S20), while design-runtime artifact mismatches persist (S21).

Finally, the *Emergence Management* subtheme emphasizes the difficulty in anticipating emergent behaviors (S79) and resolving operational conflicts caused by such behaviors (S30, S38, S69, S76). Some studies (S64, S79) point to the need for systematic approaches that can intentionally generate desired emergent properties.

4.2.2. Robust and resilient operations

This theme identifies critical vulnerabilities in maintaining SoS functionality in dynamic operational contexts. *Mission Coherence & Operational Continuity* challenges center on preserving strategic objectives during architectural transitions (S42, S37), with particular difficulties in workflow indeterminacy resolution (S50) and exception-driven adaptation (S26).

The *Resilience & Robustness* subtheme highlights four key gaps: insufficient fault-tolerance mechanisms for network or component failures (S15, S35); cascading failure risks due to operational dependencies (S11); limited environmental adaptation capabilities (S30, S45); and challenges in integrating adaptive resilience measures such as runtime monitoring and predictive maintenance (S59, S60).

4.2.3. System orchestration

This theme encapsulates the dynamic management and integration of CSs.

Under the *Dynamic & Scalable Architectures* subtheme, the evolution from static to dynamic architectures that support scalability and flexibility is emphasized (S69, S80).

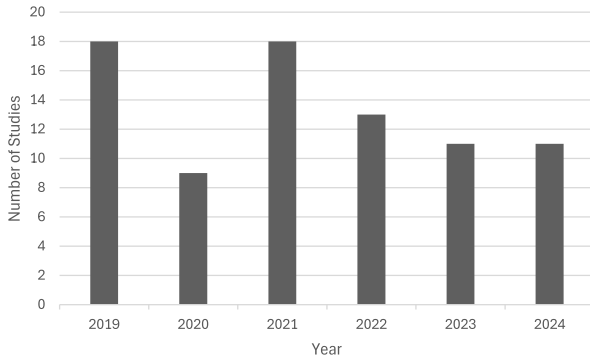
The *Dynamic Interoperability & Coordination* subtheme identifies challenges such as real-time mediator creation, standardizing communication protocols, and automating integration processes across domains (S06, S54).

The subtheme *Dynamic Discovery & Composition* addresses challenges in runtime formation and discovery of potential CSs (S22) and their functionalities (S43), as well as enabling programmatic, dynamic, and autonomous SoS construction (S46, S52, S56). S35 highlights the challenge of identifying service-providing systems while considering QoS attributes.

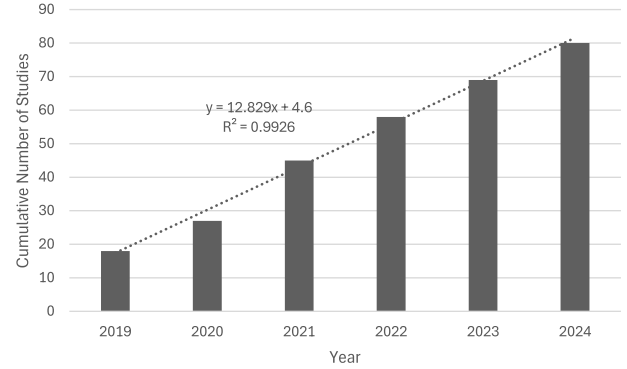
The *Interaction & Resource Management* subtheme emphasizes the need for mechanisms that facilitate autonomic inter-system interaction without prior planning (S27) and implement distributed, incremental

Table 6
Taxonomy of challenges faced in runtime composition of heterogeneous CSs for dynamic SoSs.

Themes	Subthemes	Key Challenges
Modeling and Analysis	Unified System Modeling	- Integrating quantitative and qualitative features in explicit resource allocation models (S67) - Modeling heterogeneous, multi-domain systems (S74) - Bridging micro (agent-level) and macro (organizational-level) system perspectives (S62) - Capturing emergent behavior to reveal hidden risks and patterns (S78) - Lack of model of computation (MoC) in control software (S07)
	Multi-Domain Simulation	- Developing standardized simulation methods for heterogeneous systems (S01) - Creating federated simulations that synchronize across domains and incorporate physical hardware/human interactions (S03, S47) - Enhancing metamodels to support dynamic system properties and discrete simulation (S03) - Need to simulate software architecture with various requirements (S57)
	Verification & Validation	- Minimizing verification costs in frequent reconfiguration and large model sizes (S33, S41) - Ineffective verification due to modeling inaccuracy and low knowledge-sharing among CSs (S41) - Reducing high costs and specialized knowledge requirements for physical experimental environments (S20) - Bridging the gap between design-time and runtime artifacts (S21)
	Emergence Management	- Anticipating emergent behaviors and components due to system dynamics (S79) - Developing methodologies to evaluate SoS resilience/performance early in lifecycle (S77) - Creating systematic approaches for intentionally generating useful emergent properties (S64, S79) - Resolving operational conflicts caused by undetected emergent behaviors (S30, S38, S69, S76)
Robust and Resilient Operations	Mission Coherence & Operational Continuity	- Maintaining SoS mission coherence under external constraints (S42) - Ensuring uninterrupted operations during system reconfigurations (S37) - Resolving workflow indeterminacy to enable coordinated execution (S50) - Adapting behaviors only under exceptional conditions without affecting normal operations (S26)
	Resilience & Robustness	- Enabling fault tolerance for dynamic behavior, unpredictability, and network/component failures (S15, S35) - Maintaining operational/communication independence to prevent cascading failures (S11) - Sustaining robustness amid environmental changes, faults, and adversarial conditions (S30, S45) - Implementing runtime monitoring, predictive maintenance, and adaptive resilience mechanisms (S59, S60)
System Orchestration	Dynamic & Scalable Architectures	- Evolving SoS architectures beyond static models to support dynamicity and scalability (S69, S80) - Managing continuously changing structures while balancing resilience with affordability (S37, S44, S69, S75, S77) - Supporting open, vendor-independent architectures (S63) - Balancing standardization (regulatory compliance) with flexibility (adaptability) (S66)
	Dynamic Interoperability & Coordination	- Ensuring an up-to-date SoS view while enabling interoperability for unstructured, knowledge-intensive processes (S17, S76). - Creating on-the-fly mediators for runtime composition and mediation (S54) - Establishing unified communications between CSs (S06, S07, S19, S21, S24, S54) - Automating SoS integration while reducing costs (S43, S52, S54) - Standardizing integration methods for heterogeneous systems (S16, S36, S75) - Domain-agnostic coordination (S66) by aligning communication interfaces (S12, S55) - Balancing independence with coordination requirements (S44, S51, S52, S62) - Integrating autonomous AI-enabled systems within SoS frameworks (S36, S45, S78)
	Dynamic Discovery & Composition	- Addressing runtime formation and discovery of potential CSs (S22) and their functionalities (S43) - Identifying service-providing systems while considering quality of service (QoS) attributes (S35) - Enabling programmatic, dynamic, and autonomous SoS construction (S46, S52, S56) - Supporting post-deployment composition for dynamic integration (S54) - Providing developers with assistance and automation from design-time analysis to runtime optimization (S59)
	Interaction & Resource Management	- Enabling autonomic inter-system interaction without prior planning (S27) - Implementing distributed, incremental resource scheduling across autonomous systems (S35) - Managing resources and temporal constraints in evolving environments (S50, S67)
	Control Strategies	- Balancing centralized control complexity with adherence to global objectives (S50) - Enabling real-time control and task delegation in distributed environments (S64) - Facilitating varied control strategies across independent systems (S30, S38, S46)
Heterogeneity of CSs	Component Diversity	- Physical devices (IoT, Industry 4.0, sensors, hardware) (S05, S07, S43, S56, S57, S60, S63, S70) - Digital components (abstract CSs, digital twins, multi-robot systems) (S06, S13, S65, S68) - Architectural spans (legacy vs. modern frameworks) (S02, S23, S27, S58, S76)
	Representation Fragmentation	- Incompatible data formats, sources, and models (S04, S05, S14, S44, S47, S52, S54) - Semantic mismatches in representations (S34, S48) - Validation gaps between abstract and concrete implementations (S06, S13, S21, S59, S60)
	Proprietary Constraints	- Vendor lock-ins and proprietary solutions (S23, S34, S54, S63, S75) - Black-box components and defiant systems (S26, S34) - Outdated protocols and legacy architectures (S23, S58)
	Operational Coupling	- Cascading compatibility issues (S16, S29, S49) - Mediation complexity in bridging systems (S05, S43, S65) - Security vulnerabilities in heterogeneous networks (S39) - Operational fragility from undocumented interactions (S23, S58)



(a) Number of studies published per year.



(b) Cumulative number of studies published per year.

Fig. 3. Publication trends of the studies selected for the SLR (2019–2024).

resource scheduling across autonomous CSs (S35). Some studies highlight the challenge of managing resources and temporal constraints in evolving environments (S50, S67).

Finally, the *Control Strategies* subtheme highlights the need to balance centralized control complexity with adherence to global objectives (S50), enable real-time control and task delegation in distributed environments (S64), and facilitate varied control strategies across independent CSs (S30, S38, S46). This balance is particularly challenging in integrating autonomous AI-enabled CSs within SoS frameworks (S36, S45, S78), where the goal is to align autonomous decision-making with global SoS objectives while preserving system independence.

4.2.4. Heterogeneity of constituent systems

This theme addresses the multidimensional integration challenges arising from technical, organizational, and historical diversity in CSs of SoSs components. The literature identifies four interconnected dimensions of heterogeneity that impede runtime composition, as outlined below.

Component Diversity manifests through incompatible IoT ecosystems (S70), Industry 4.0 devices (S63), and specialized hardware (S05, S07) that resist standardized integration. Studies highlight particular difficulties in bridging abstract CS specifications (S06, S13) with physical implementations, especially when digital twins (S68) or multi-robot systems (S65) introduce additional interface layers. This diversity compounds architectural tensions between modern frameworks (S58) and legacy systems (S23, S58).

Representation Fragmentation emerges from incompatible simulation models (S04, S47) and data formats (S52, S54), with studies emphasizing the cascading effects of custom information models (S34). The subtheme identifies critical gaps in toolchain interoperability (S59) and validation mismatches between design-time models and runtime behaviors, particularly in human-AI environments (S60).

Proprietary Constraints create systemic rigidity through vendor lock-ins (S63, S75) and black-box components (S34) that obscure operational logic. Studies highlight how bespoke protocols (S23, S34) and defiant components (S26) actively resist integration efforts, while legacy architectures (S23) impose outdated communication patterns that conflict with runtime composition requirements.

Operational Coupling reveals three compounding effects: 1) Networked dependencies (S29, S49) that propagate failures across operating systems (OS) or software-intensive SoS (SiSoS) architectures (S02, S27), 2) Secure communication impeded by protocol heterogeneity (S39), and 3) Exponential mediation costs when scaling deployments (S05, S43). Studies particularly emphasize the fragility of systems combining modern and legacy components (S16, S58).

4.3. Solutions (RQ₂)

Table 7 presents the taxonomy of solutions proposed to enable runtime composition in dynamic SoSs. These solutions address various aspects of interoperability, integration, and reconfiguration of CSs.

4.3.1. Simulation, modeling, and design

This theme encompasses approaches for representing, simulating, and designing SoSs, focusing on modularity and interoperability.

Solutions in subtheme *Co-Simulation & Integrated Simulation* create interfaces between different simulation platforms and testbeds (e.g., SAGM Toolbox with Mosaik; S18), and employ frameworks such as SEAT (S08, S36) to support modular, application programming interface (API) driven simulation.

Some studies demonstrate diverse *Modeling* techniques, ranging from hypergraph-based representations (S11) to scenario-based frameworks (S26), with generic metamodels (S55) abstracting internal details to prioritize communication properties and emergent behaviors.

The *Lifecycle Management* approaches address end-to-end SoSE through process-oriented frameworks that span design, operation, and evolution. For example, SoS lifecycle management (SoSLM) (S24) integrates Product Lifecycle Management (PLM) with industrial internet of things (IIoT) for coordinated governance in distributed environments. Meanwhile, the SoSE Lifecycle Concept (S10) introduces a multi-stage, agile methodology for smart product-based systems.

Digital Twins couple physical and virtual systems for real-time monitoring (S61) and dynamic reconfiguration (S40). *Design-time Approaches* like ADORE (S38) embed adaptation policies into runtime workflows, while S73 leverages knowledge bases with stochastic search optimization to generate adaptation strategies.

4.3.2. Semantic approaches

This theme focuses on enhancing system understanding and interoperability through semantic technologies.

Mission and Domain-Focused Ontology Design approaches, such as OSysRec (S74), address model mismatches by synthesizing structural, dynamic, and management aspects into resource description framework (RDF) based linked data models.

Ontology-Driven System Composition and Discoverability techniques leverage ontologies for dynamic discovery and composition, employing holonic architectures and natural language processing (NLP) to generate CS descriptions (S32, S46).

4.3.3. Integration approaches

This theme aims to provide seamless, scalable, and domain-agnostic connectivity among CSs.

Table 7
Taxonomy of solutions.

Theme	Subtheme	Key Points
Simulation, Modeling, and Design	Co-Simulation & Integrated Simulation	• Adaptations from Smart Grid to Industry 4.0 for CS simulation and emergent behavior detection (S01) • Cloud-based co-simulation using HLA standard and Kubernetes (S48) • Interface between SAGM Toolbox and Mosaik (S18) • SEAT framework: Modular, cloud-enabled, API-driven simulation testbed (S08, S36) • DEDSM for integrating C model with C2 model (S03) • MBSE and DEVS integration via SoSADL for simulation/model transformation (S57) • Behavioral model of interoperability implemented in the agent-based simulation program MANA (S49) • LEGO-robot-based physical exemplar for platooning and sensor/actuator data collection (S20)
	Modeling	• Hypergraph-based modeling of SoS internal structures (S11) • Scenario-based modeling for normal and exceptional conditions using AOP wrappers (S26) • Escher: exposing only external behavior via a refinement-aware message bus (S09) • Abstraction of internal details to focus on communication properties and behaviours (S55) • A generic metamodel in Maude to control resources and behavioral features of SoS (S67)
	Design-time Approaches	• ADORE: integrates Cognitive Work Analysis and Goal-Oriented Requirements Engineering to model domain analysis, collaborative tasks, and scenarios requiring adaptation strategies (S38) • Design-time adaptation strategies seeded from a knowledge base and optimized via stochastic search (S73)
	Digital Twins	• Digital Twins for coupling the physical and virtual worlds in SoS architecture descriptions (S22) • Real-time simulation and monitoring with Digital Twins (S61) • Embedding Digital Twins in synthetic environments for scenario simulation (S78) • Leveraging digital twins for real-time simulation and monitoring (S60) • Use of IoT technology and Digital Twins to enable dynamic reconfiguration (S10) • Modeling CSs as DTs for runtime adaptation (S40)
	Lifecycle Management	• SoSLM: Process-oriented lifecycle management integrating PLM and IIoT methodologies (S24) • SoSE Lifecycle Concept: Multi-stage, bidirectional approach for smart product-based systems (S10)
Semantic Approaches	Mission and Domain-Focused Ontology Design	• OSysRec ontology for synthesizing structural, dynamic, and management aspects of system reconfiguration (S74) • RDF-based common models addressing model mismatches (S34) • Ontology describing domain semantics and logical analysis (S05, S31) • Linked data and ontologies for data representation and exchange (S17) • Knowledge Intensive Process Ontology and Notation for interoperability decisions (S76)
	Ontology-Driven System Composition and Discoverability	• Utilizing ontologies to represent systems, enabling them to discover, understand, and compose at runtime (S27, S46) • Utilizing the holon ontology to describe device properties and functionalities (S43, S32, S46) • NLP-generated holon descriptions from publicly-available device data (S32) • Holonic architecture enabling systems to discover, understand, and compose at runtime (S46)
Integration Approaches	Framework-Based Integration	• Conceptual Integration Framework (integrating model, tool, and execution platforms) (S47) • Federation of rule engines for rule control and integration policies (S34) • ISOs framework to enhance conceptual interoperability (S04) • Use of ontologies and holons for enabling interoperability (S46) • Using blockchain to enable trusted interoperability (S64)
	Protocol-Driven Integration	• 5G-enabled low-latency and peer-to-peer integration for heterogeneous CSs (S39) • JSON-based protocols for CS interactions (S45)
	Domain-specific Integration	• Integration of model-based simulations with domain-specific support (S44) • Using DSLs to cover domain-wise integration space (S56) • ROGER framework abstracts away low-level details of CSs using domain-specific language IFP (S13) • Seamless Integration Framework for domain-wide standardization (S75)
Architectural Approaches	Adaptive & Dynamic Architectures	• dynADL: a lightweight ADL for dynamic architectures supporting runtime reconfiguration and component management (S69) • Enhanced SoSADL with fuzzy constructs based on Fuzzy Theory (S22) • A metamodel and enterprise architecture for dynamic architecting of SoS configurations (S44) • Emergence-aware architecture description method for improved design explainability (S79) • Game-theory-based multi-domain design (S29) • Publish-subscribe architectures for scalability, integration, and interoperability (S51, S23) • Transitioning between architectures in SoSs to dynamically adapt to mission requirements through iterative re-configuration (S80, S27)
	Service-Oriented Architectures	• Microservice-driven similarity analysis and GraphQL-based API unification (S06) • MSA for modularity, scalability, and resilience (S60) • Autonomous aggregator architecture within an SOA for smart buildings with multi-criticality policies and self-X properties (S70) • Eclipse Arrowhead Framework (S05, S16, S52, S12, S21, S58, S19, S63)
	Middleware & Communication	• Architecture using TSN, MPLS, two-step admission control, distributed service discovery, and cloud resources for on-demand service instantiation (S35) • Three-tier architecture using Discovery and Access Broker (DAB) with RM-ODP (S14) • MediArch: three-layer mediator for interaction and control (S30) • Hetero-Genius: middleware for runtime discovery and composition (S54) • Employing Discrete Controller Synthesis (DCS) to automatically generate controllers for coordination (S66)

Table 7
Continued

Theme	Subtheme	Key Points
Formal & Analytical Methods	Verification & Analytical Strategies	• Verification strategies for structural/data correctness and dynamic interactions (S55) • Formal analytical techniques (graph theory, multi-attribute utility theory) for mission path evaluation (S31)
	Hybrid Approaches	• TRC-MM with RT-Maude for time-resource aware missions (S50) • Continuous Verification via MAPE-K Patterns with Model Slicing (S33, S41) • Maude Strategy Language for dynamic reconfigurations (S42)
Resilience & Fault Tolerance	Architectural Reconfiguration	• ReViTA framework for fault tolerance via architectural reconfigurations (S15) • CSSAD method for reconstructing SoS to restore lost operational capabilities (S28)
	Hybrid Reconfiguration	• Bottom-up fault detection with top-down reconfiguration for enhanced resilience (S11) • Operation-loop-based mission reliability evaluation model for external shocks and topology reconfiguration (S53)
	Nature-Inspired Approaches	• Bio-inspired approach using ecological fitness principles and the DoSO metric for resilience and affordability (S77) • Mosaic control system enabling agents for resilient adaptation (S29) • Multi-swarm-based cooperative reconfiguration model to maximize the resilience (S71)
	AI-driven Approaches	• DRLRESF that uses GCNs and PPO for autonomous learning of resilience strategies (S72) • Dynamically generating adaptation strategies in uncertain environment (S73)
Cross-Cutting Approaches		• Distributed Cognitive Toolkit (language-agnostic extension of CST for distributed cognitive systems) (S45) • Integration of domain-specific architecture models with multi-agent systems for decentralized and adaptive CPS (S62) • CorteX framework: an interoperable, modular, and extensible decentralized control system with a self-describing data model, designed for pub/sub, service-oriented applications, and demonstrated in real-world fusion hardware (S02, S65) • Service-oriented approach for enabling interoperability between SOSJ and non-SOSJ through OPC UA (S07) • Portable data connectivity leveraging OPC-UA to encapsulate digital twins, ensuring semantic interoperability and decentralized and modular interactions for dynamic reconfiguration (S68) • The CERBERO Project: Integrates modeling, deployment, and verification tools to enable a multi-level self-adaptation infrastructure with a hierarchical adaptation loop for CPS, supporting functional, non-functional, and repair-oriented adaptivity (S59)

Framework-Based Integration solutions such as infrastructure SoS (ISoS) (S04) and Holons (S46) offer high-level architectures for integrating CSs.

Protocol-Driven Integration subtheme focuses on technical enablers for runtime composition, employing lightweight data formats like JavaScript object notation (JSON) (S45) and emerging technologies like 5G (S39) to address challenges of heterogeneity and latency.

Domain-Specific Integration methods use domain-specific languages (DSLs) or metamodels to facilitate tighter coupling and reduced ambiguity for CSs operating within similar contexts.

4.3.4. Architectural approaches

This theme comprises solutions dedicated to constructing adaptable and scalable SoS infrastructures.

Studies on *Adaptive & Dynamic Architectures* emphasize formal methods, such as lightweight ADLs, including dynaADL (S69) and fuzzy-theory-enhanced SosADL (S22), to address runtime reconfiguration and uncertainty. Metamodels for enterprise architecture (S44) and emergence-aware methods (S79) contribute to design explainability, while game-theory applications (S29) introduce mathematical rigor into multi-domain challenges. Practical patterns like publish-subscribe architectures (S51, S23) address scalability and interoperability, whereas architectural transitions enable SoS to dynamically adapt to evolving mission requirements through iterative reconfiguration (S80, S27). These approaches reflect a cross-disciplinary integration that balances theoretical innovation with practical applicability.

The *Service-Oriented Architectures* (SOA) subtheme highlights the role of SOA in enabling modular, scalable, and resilient runtime-composable SoSs. Research spans specialized service implementations, such as microservice architectures (MSA) with GraphQL-based API unification (S06, S60) and domain-specific implementations (S70). It also includes standardized frameworks, exemplified by the widespread adoption of the Eclipse Arrowhead Framework (EAF) (S05, S16, S52, S12, S21, S58, S19, S63).

The subtheme of *Middleware & Communication Architectures* emphasizes real-time coordination, scalability, and dynamic resource management. It covers multi-layered approaches, such as three-tier architectures with Discovery and Access Brokers (DAB) (S14) and frameworks like MediArch (S30). Hybrid solutions integrate Time-Sensitive

Networking (TSN), Multiprotocol Label Switching (MPLS), and cloud resources (e.g., S35), while middleware frameworks like Hetero-Genius (S54) and Discrete Controller Synthesis (DCS) (S66) provide automated discovery, composition, and adaptive coordination.

4.3.5. Formal and analytical methods

This theme presents frameworks to ensure mission-critical systems' adaptability, correctness, and efficiency.

Verification Strategies and Foundational Techniques involve methods like verification strategies (S55) and foundational theories (e.g., graph theory, multi-attribute theory (S31) to ensure analytical rigor in mission path assessment.

Hybrid Approaches focus on integrating Time-Resource Aware Control metamodels with tools such as RT-Maude (S50) and leveraging continuous verification methods—e.g., using Monitor-Analyze-Plan-Execute over Knowledge (MAPE-K) patterns and property-based model slicing (S33, S41)—to balance static verification with dynamic control. Rewriting logic via the Maude Strategy Language (S42) also supports flexible specification of dynamic reconfigurations.

4.3.6. Resilience and fault tolerance

This theme consists of approaches that maintain SoS functionality amid failures and disruptions during dynamic composition with other CSs.

A key trend is the development of *Architectural Reconfiguration Frameworks* like Reconfigurations Via Transient Architectural Configurations (ReViTA) (S15) for fault-tolerant design and CSoS autonomous decision-making (CSSAD) (S28) for post-disruption capability restoration. These frameworks work alongside *hybrid reconfiguration* models, such as bottom-up fault detection paired with top-down recovery (S11) and operation-loop-based models for mission reliability under external shocks (S53).

Some studies also highlight *Nature-Inspired Principles*—ecological fitness metrics (DoSO in S77) and decentralized strategies (mosaic agents in S29, multi-swarm reconfiguration in S71)—to balance with cost-effectiveness.

AI-Driven Techniques enable autonomous and context-aware resilience by combining graph networks with reinforcement learning (RL)

to infer resilience strategies (S72), and dynamically generating adaptation policies for uncertain environments (S73). These approaches illustrate a paradigm where proactive design and reactive adaptation converge to enhance system robustness in complex operational contexts.

4.3.7. Cross-cutting approaches

A subset of the reviewed solutions spans multiple themes, combining architectural, semantic, and integration strategies. For example, the CorteX framework (S02, S65) offers an extensible, decentralized control system with a self-describing data model, built on publish-subscribe and service-oriented principles. Similarly, the CERBERO Project (S59) integrates modeling, deployment, and verification into a multi-level self-adaptation infrastructure for cyber-physical systems (CPS). OPC UA exemplifies protocol-driven integration (S07) while extending into digital twins (S68), illustrating its cross-cutting role in semantic and architectural interoperability. Finally, advanced cognitive and multi-agent strategies, as seen in the Distributed Cognitive Toolkit (S45) and integrating domain-specific models with multi-agent systems (S62), illustrate emerging directions in the research subject.

4.4. Tooling ecosystem (RQ₃)

Our synthesis reveals a diverse set of tools and technologies supporting runtime composition, interoperability, and dynamic reconfiguration in SoSs. We organized these tools into themes and subthemes, reflecting distinct but complementary approaches to SoS design and operation (Table 8).

Simulation & Scenario Testing tools dominate the ecosystem, ranging from co-simulation platforms like Mosaik (S01, S18, S47) to high-fidelity simulators such as Gazebo (S40, S51) and Unity (S40, S48), which validate dynamic behaviors in domains like drone control (VENTOS; S33) and urban mobility (SUMO; S48). These are complemented by *Formal Modeling & Verification* tools like PRISM (probabilistic verification; S33, S41) and SysML (model-driven design; S21, S37), which ensure rigor in interoperability and safety.

Semantic Technologies—such as Protégé (S27, S31, S46) and RDF-based approaches (S04, S17, S34)—alongside communication middleware like Open Platform Communications-Unified Architecture (OPC UA) (S07, S12, S68) and Message Queuing Telemetry Transport (MQTT) (S10, S16, S48), reflect efforts to standardize cross-domain data exchange and promote interoperability.

The *Eclipse Ecosystem* stands out as a major contributor, with integrated frameworks and tools including EAF, Hono, HawkBit, Ditto, Kapaia, and Corto appearing across 11 studies (S05, S07, S12, S19, S21, S22, S24, S44, S52, S58, S63). Its widespread adoption highlights the growing demand for cohesive, extensible solutions for industrial SoS development.

Other technologies support deployment and interaction across physical and digital layers. *Containerization* platforms like Docker and Kubernetes (S14, S45, S48) enable modular and agile deployment, while *Digital Twin* frameworks (e.g., CONTACT Elements; S10, S60) facilitate runtime monitoring and adaptation. Similarly, *Robotic and IoT Platforms* such as ROS (S65, S68) help bridge hardware-software integration in real-world settings.

Well-established tools like EAF, MQTT, ROS, and Docker reflect maturity for real-world deployment. In contrast, AI or machine learning (ML) techniques—including fine-tuned bidirectional encoder representations from transformers (BERT) (S32), Deep RL (DRL) (S72), and Neural Networks (S11)—are underutilized, signaling their emerging status rather than integration maturity. Similarly, blockchain (Orthus; S64) and low-code tools (Node-RED; S16) lack cohesive adoption, operating in isolation rather than as part of unified toolchains.

Despite the rich ecosystem, tool integration remains fragmented. While platforms like EAF (S05, S63) enable service orchestration and DIME (S56) supports low-code integration, few studies demonstrate complete toolchains spanning from high-level modeling to simulation to

runtime adaptation (e.g., NVIDIA Omniverse; S68). This absence highlights a gap in holistic SoSE environments capable of supporting the whole system lifecycle.

4.5. Evaluation (RQ₄)

This section explores how studies addressing runtime composition in SoSs evaluate their proposed solutions. We present our findings organized into two complementary aspects: 1) the methodologies employed to assess solutions (Section 4.5.1), and 2) the application domains and case studies contextualizing these evaluations (Section 4.5.2).

4.5.1. Evaluation methods

Table 9 provides the taxonomy of the evaluation methods employed in the proposed solutions. These themes and their respective subthemes are elaborated in the remainder of this section.

Simulation-based evaluation. These methods dominate the landscape, likely due to their ability to model complex interactions without incurring the substantial resource demands of full-scale implementations. Among these methods, *Scenario-Based Simulation* is the most prevalent category, with researchers generating various scenarios to evaluate performance, develop adaptation strategies, and validate reconfiguration strategies. In addition, *Autonomous Systems Simulation* forms a distinct cluster (8 studies), reflecting the growing importance of autonomous systems in SoS contexts. These ranged from unmanned aerial vehicle (UAV) swarm reliability assessments using Monte Carlo methods (S53) to multi-swarm reconfigurations under stack scenarios (S71). *Co-Simulation* enables the integration of different simulators to study interactions between CSs in domains such as industrial automation (S01) and electric vehicle (EV) fleet management (S48). *Agent-Based Simulation* models CSs as autonomous agents, enabling iterative design and strategic analysis of emergent behaviors (S62, S04). *Discrete-Event Simulations* capture SoS responses under specific events or adverse conditions (S29), providing insights into resilience and adaptability.

Implementation-based evaluation. This is the second most prevalent approach, emphasizing real-world applicability and feasibility. Most studies developed prototypes (12 studies) and proof-of-concepts (10 studies) to validate the technical feasibility of their solutions. Hardware-in-the-Loop testing was particularly prominent within *Testbed* implementations, enabling evaluation under conditions closely approximating real-world operational environments.

Human-centered evaluation. Although less frequently employed, these methods are critical for evaluating the usability and stakeholder acceptance of SoS solutions. These approaches include user studies with non-specialists (S74, S79), developer-focused evaluations (S54), expert assessments via techniques like the Architecture Trade-off Analysis Method (S34), and structured interviews (S15), and collaborative evaluation techniques including multi-stakeholder workshops (S74).

Formal and static analysis. These methods provide rigorous verification capabilities through *model checking* (S42, S50) and *formally verified implementations* (S33, S55). They appeared in relatively few studies, suggesting potential for expanded application in future research.

4.5.2. Application domains and case studies

Most studies situate their evaluation activities within well-defined application domains and real-world case scenarios. This contextual grounding is essential for assessing the practical relevance and transferability of the proposed solutions. Therefore, we systematically mapped the case studies and application domains across the selected studies.

As shown in Fig. 4, our analysis reveals a diverse ecosystem of domains where SoS approaches are deployed, from industrial automation to emergency response. In this figure, shaded nodes represent

Table 8
Tooling ecosystem for runtime composition in dynamic SoSs.

Theme	Subtheme	Tools With Study IDs
Simulation & Scenario Testing	Co-Simulation Platforms	Mosaik (S01, S18, S47), Portico RTI (S48)
	Dynamic & Fuzzy Model Simulation	MS4ME Platform (S57), DEVS-Suite (S22), DEVSim + + (S03)
	High-Fidelity Simulations (drones, urban mobility, etc)	Gazebo (S40, S51), Unity (S40, S48), NVIDIA Omniverse (S68), SUMO (S48), Unreal Engine (S36), Dash (S40), Dragonfly Simulator (S26), VENTOS Simulator for platooning (S33)
	Network Simulation	NS3 (S02), OMNeT + + /Mesosaurus (S46)
	Agent-based Simulation	MANA (S49), JADE (S80)
	Distributed Simulation	DIS/RPR FOM (S04)
Formal Modeling & Verification	Model-Driven Design & Development	SysML (S21, S37), SGAM (S18, S61, S75), PREESM (S59), MagicDraw (S21), Enterprise Architect (S18)
	Formal Verification	Papyrus (S55), PRISM (S33, S41), Uppaal (S55), Maude/MaudeSL (S42, S50, S67), SMC (S22, S41), LTSA (S41)
	Scenario Modeling & Language Support	VisualFML (S22), MSCs (S26), FisPro (S22), MOF (S44), DSLs (S44), CityGML (S44), MTSA (S66), Antlr (S69), WebGME (S47), PLASMA (S22)
Semantic Technologies	Ontology Development	Protégé (S27, S31, S46), RDF (S04, S17, S34), RDFS (S04), OWL (S04), Turtle (S17), SPARQL (S17), SysML (S74)
	Ontology Conversion	OWL2UML (S27), OWL2GRA (S27)
	Data Exchange	JSON-LD (S58), XML (S65)
	Centralized Supervision	CoDAMOS ontology (S46, S54)
Digital Twin & Containerization	Digital Twin	CONTACT Elements for integration (S10), Java EE Environment for deployment (S60)
	Containerization	Docker (S14, S45, S48, S58), Kubernetes (S14, S45, S48), Docker Compose (S64)
Communication Middleware	Industrial Protocols	OPC UA (S07, S12, S68), FreeOpcUa/open62541 (S68), RAMI 4.0 (S75)
	Pub/Sub Systems	MQTT (S10, S16, S36, S48, S52, S54), Mosquitto MQTT (S51), DDS (S56), RabbitMQ (S64), AMQP (S17), NSNotificationCenter (S23), MSMQ (S23), EMQX (S36)
Extended Integration	Integration	HLA/RTI (Portico) (S02, S03, S47)
	Service Discovery	Protocol Buffers (S23), DNS/DHCP (S35), MarkLogic Server (S14), THREDDs (S14)
Eclipse Ecosystem	Core Frameworks	Eclipse Arrowhead (S05, S12, S19, S24, S52, S58, S63), Arrowhead Client Library (S63)
	Device Management	Eclipse Hono (S07, S12, S19), Eclipse hawkBit (S19), Eclipse Ditto (S19), Eclipse Kura/Kapua (S19), Eclipse Vorto (S19), Eclipse 4diac™ (S12), Eclipse Milo (S07)
	Model-Driven Development	Fuzzy SosADL Studio (S22), Arrowhead Management Tool (S21), VIATRA (S21, S44), Sirius Framework (S44)
Architecture & Development	Architectural Modeling	UPDM (S37), SosADL (S30, S57), ArchSoS Language (S42)
	Orchestration	ESB (S60)
	Architecture Frameworks	NATO AF (S74), MAF (S75)
Robotic & IoT Platforms	Robotic Platforms	LEGO MINDSTORMS EV3 (S20), ROS (S65, S68), POSIX (S68), NVIDIA Isaac Sim (S68)
	IoT Devices	Raspberry Pi-based systems (S70, S68)
Data Management, Cloud & Infrastructure	Data Flow	SPIDER (S59)
	Data Optimization	ElasticSearch/OpenSearch (S14)
	Cloud Provisioning	MITRE Symphony (S36), Terraform (S36), Ansible (S36)
Workflow, Low-Code & Testing Tools	Workflow Execution	Node-RED (S16)
	Low-Code & Integration	MicrographQL Tool (S06), StringTemplate (S18), DIME (S56)
	Testing & Modeling	MATLAB Fuzzy Logic Toolbox (S22), AspectJ (S26), Prolog Rule Engines (S34), mKAOS (S15), W3C WoT (S58), IoT Broker (S60), Orthus Blockchain (S64), Apache JMeter (S64)
AI/ML	AI/ML Frameworks	Fine-tuned BERT (S32), AI/ML for data processing (S39), Machine learning for optimization (S40), Deep RL (S72), Neural Networks (S11)

case studies, while unshaded nodes indicate application domains. Notably, some application domains lack associated case studies, such as Satellites (S04), Aerospace (S13), and Industry 5.0 (S68), as these studies mention a domain but do not evaluate their approach with a case study. Furthermore, some studies assess their approach using multiple case studies (S66). Among these domains, industry, defense, and disaster management are the most prominent application areas, highlighting their critical role in addressing complex, real-world challenges.

5. Discussion

This section synthesizes the results, explores research gaps and potential future directions, and discusses the implications for research and practice.

5.1. Key findings and interpretations

This section presents key findings of the results, their interpretation, and critical analysis, and situates them within the existing literature.

Table 9
Taxonomy of evaluation methods.

Methods	Subthemes	Evaluation methods with corresponding study IDs
Simulation	Co-Simulation	• Co-simulation of modular production units in industrial automation using Mosaik (S01) • Co-simulation of EV fleet ecosystems for energy/charging/infrastructure load analysis (S48) • Simulation of C3 federation (S03) • Simulation for varying interoperability parameters in SAR missions (S49)
	Scenario-Based Simulation	• For validating reconfiguration strategies and SoS agility (S11, S25, S34, S37) • To generate scenarios for evaluating performance (S42, S44, S41, S35, S28, S39, S17, S77, S66, S33, S58) • Scenario-based simulation for adaptation strategy generation (S26, S73)
	Agent-Based Simulation	• Iterative design and performance analysis using agent-based models (S62) • Multi-actor strategic behavior analysis using game-theoretic value modeling (S04)
	Discrete-Event and Numerical Simulation	• Evaluating performance of autonomous systems under adversarial conditions (S29) • OMNeT++ simulation for Mesos cluster scaling (S46) • Formal simulation of fuzzy SoS architectures using DEVS-Suite/PLASMA (S22) • DEVS models in MS4ME environment with ASAS method (S57) • Maude-based rewriting logic simulation for resource allocation (S67)
	Autonomous Systems Simulation (UxV/UAV/UGV)	• NetLogo/ESimP in SEAT execution environment (S08) • Gazebo/DroneResponse for sUAS (S51) • JADE-based UVF coordination (S80) • Monte Carlo simulation for UAV swarm reliability and failure response (S53) • Factory simulation of autonomous transport vehicles for safety evaluation (S61) • Multi-swarm reconfiguration under attack scenarios (S71) • Simulating UWSoS attack scenarios with varying levels of disruption (S72) • Hybrid cloud simulation for mission scenarios (S36)
Implementation-Based Evaluation	Testbeds	• Hardware-in-the-Loop Testing (S02, S20, S07, S56, S46, S65, S68) • Benchmarking accuracy/scalability of tools/algorithms (S32)
	Proof-of-Concepts	• Validation of technical feasibility (S06, S13, S21, S27, S31, S47, S52, S55, S70, S66)
	Prototype Implementations	• Functional implementation for practical validation (S12, S14, S16, S23, S59, S63, S60, S64, S69, S24, S17, S30)
Human-Centered Evaluation	User Studies	• Interviews with non-specialists (S74, S79) • User experiments (S43) • Developer user study (S54)
	Expert Assessment	• Semi-structured interviews (S15) • Architecture Trade-off Analysis Method (ATAM) (S34) • Expert questionnaire evaluation (S76)
	Collaborative Evaluation	• Discussion-based evaluation (S47) • Multi-stakeholder workshops (S74)
Formal & Static Analysis	Formal Analysis	• Model checking (S42, S50) • Formally verified implementations (S33, S55)
	Static Analysis	• Graph-based methods for resilience, robustness, and inter-agent communication analysis (S62)

5.1.1. Fragmented research communities

The fragmented distribution of research across diverse venues (Section 4.1.1), with limited overlap, underscores both the challenges and opportunities inherent in the research subject. While this fragmentation reflects an emergent, interdisciplinary field shaped by diverse perspectives and methodologies, it risks redundant efforts and stalled progress due to limited cross-community collaboration. By systematically connecting these siloed discourses, the SLR addresses the immediate need for synthesis to mitigate fragmentation and highlights the potential for convergence, fostering interdisciplinary dialogue and accelerating innovation in this evolving domain.

5.1.2. The challenge landscape of runtime composition in SoSs

The challenges to enabling runtime composition in SoSs (RQ₁) range from technical interoperability concerns to architectural governance issues.

Multi-dimensional heterogeneity. The heterogeneity in SoS extends well beyond technical incompatibility: it also encompasses proprietary restrictions and operational coupling that introduce systemic rigidity, posing a significant barrier that requires interdisciplinary solutions. We discovered (Section 4.2.4) that reconciling differences in physical devices, data formats, communication protocols, and development

methodologies is crucial for ensuring seamless integration, strong performance, and flexible operations. We believe that this issue of heterogeneity is one of the key factors that necessitate moving beyond traditional approaches, as highlighted by (Fang, 2022).

The modeling-reality gap. Our analysis reveals a persistent gap between design-time models and real-world SoS behavior, particularly in capturing emergent properties (Section 4.2.1) and multi-domain interactions. This gap aligns with the difficulty in managing architectural emergence and integration (Fang, 2022). Our taxonomy contextualizes these challenges regarding runtime composition and reveals why traditional verification and validation remain problematic, even with advanced models. The exponential increase in verification costs during frequent reconfigurations (S33, S41) highlights the inadequacy of static techniques for dynamic SoSs. These findings underscore the need for unified, lightweight frameworks that enable continuous validation and close the gap between modeling assumptions and runtime realities.

The autonomy-coordination paradox. Evident in conflicts between decentralized control strategies (S50) and mission coherence requirements (S42), this reflects a core tension in *System Orchestration* theme (Section 4.2.3). Solutions like hybrid architectures (S69) and Discrete Controller Synthesis (S66) attempt to balance these priorities but lack

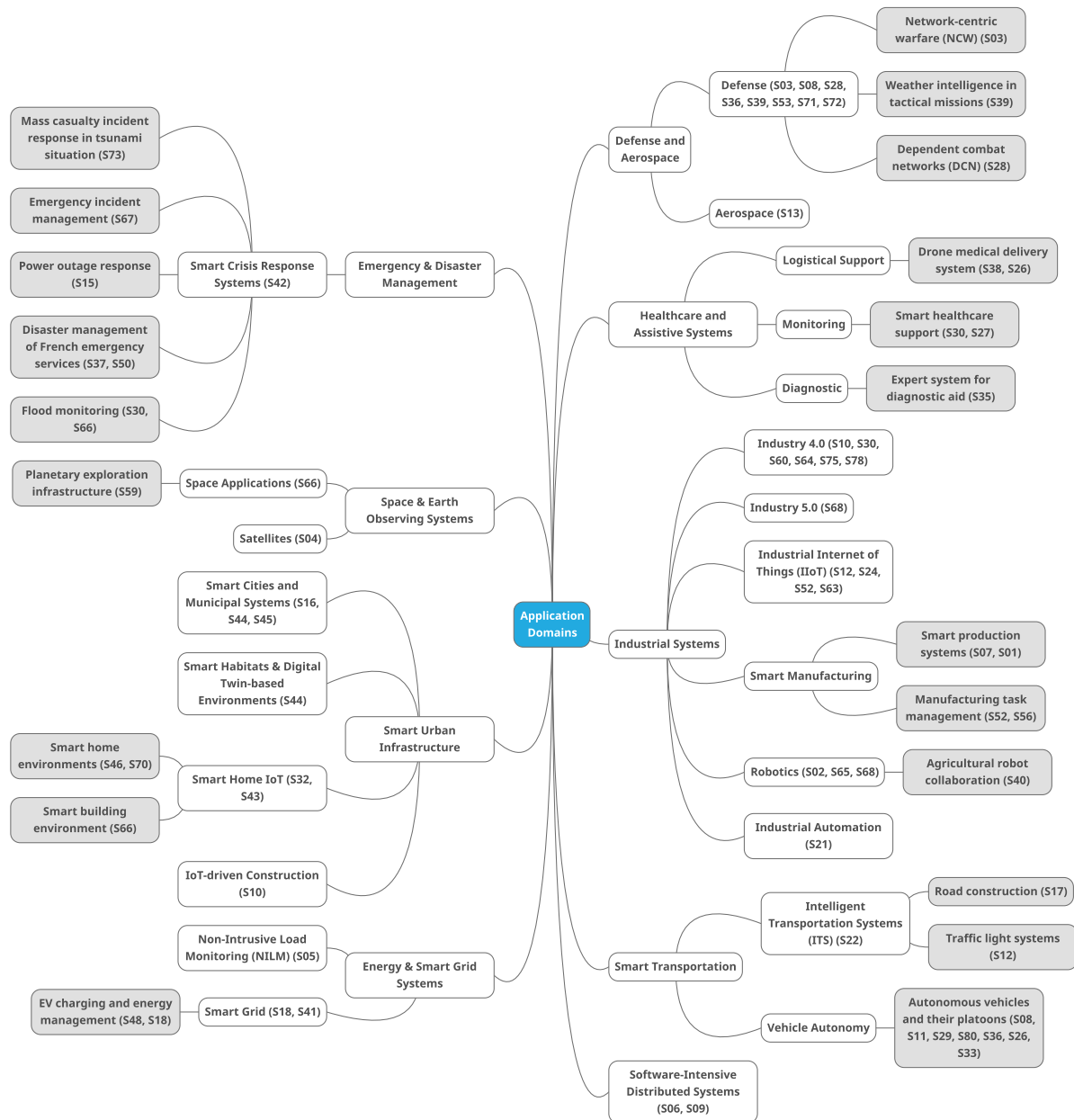


Fig. 4. Application domains identified in the selected studies. Unshaded nodes represent domains and subdomains, while shaded nodes indicate case studies.

standardized evaluation metrics. This paradox appears across multiple subthemes such as *Dynamic Discovery & Composition*, *Control Strategies*, and *Mission Coherence & Operational Continuity*. These limitations become especially problematic under volatile environmental conditions or component-level faults, where miscoordination can threaten operational continuity or trigger cascading failures. Effective solutions must embrace this paradox by developing adaptive strategies that maintain mission coherence while balancing autonomy, resilience, and context-aware coordination.

5.1.3. Evolution of solution approaches

Our analysis of solutions (Section 4.3) reveals several notable trends in addressing the identified challenges. While diverse solutions exist, the landscape is fragmented, with SOA being the noteworthy excep-

tion. Additionally, we observe a fundamental tension between achieving domain-specific precision and enabling cross-domain scalability.

The shift toward adaptive architectures. Table 7 demonstrates an evolutionary trajectory from static to dynamic architectural approaches. Early solutions focused on standardized interfaces and protocols (S04, S45), while more recent approaches emphasize adaptive architectures (S69, S22) and dynamic reconfiguration capabilities (S80, S27). This evolution reflects growing recognition that static interoperability solutions cannot adequately address the inherent dynamism of modern SoS environments (Maciel et al., 2024).

The emergence of hybrid approaches that combine formal methods with dynamic adaptation mechanisms (S69, S22, S55, S42) is particularly noteworthy. These hybrid approaches suggest a maturing field

moving beyond the false dichotomy of formal rigor versus adaptive flexibility.

This shift aligns with observations by Fang (2022), who noted the diminishing relevance of traditional architectural approaches where systems autonomously self-organize without centralized control. Our findings extend this observation by identifying emerging architectural patterns to address these dynamic contexts, particularly in runtime composition scenarios.

Convergence around service-oriented paradigms. The convergence around SOA approaches, with particular emphasis on microservices (S06, S60) and standardized frameworks like the EAF (S05, S16, S52, S12, S21, S58, S19, S63), suggests an emerging consensus around their suitability for SoS integration challenges.

These findings align with Garcés et al. (2021), who identified SOA and MSA among key architectural patterns for addressing SoS requirements. However, our analysis goes deeper by revealing the prevalence of these patterns and their specific implementations and adaptations for runtime composition contexts.

This convergence may also indicate a consolidation phase in the field, where researchers build upon established architectural patterns rather than propose fundamentally new approaches. However, it also raises concerns about potential limitations in addressing challenges outside the service-oriented paradigm, such as physical integration and emergent behaviors in CPS contexts.

The rise of semantic approaches. Semantic technologies, particularly ontology-based approaches (S32, S46, S74), address dynamic discovery and interoperability in SoS contexts by enabling semantic understanding of capabilities and constraints. These technologies reflect a shift beyond syntactic interoperability, aligning with emphasis on ontology alignment and linked-data inference (Nilsson et al., 2024). Our analysis highlights concrete implementations in SoS, such as integrating ontologies with holonic architectures (S46) and service discovery (S32), demonstrating the complementary use of semantic and architectural solutions.

These findings extend the cognitive interoperability framework for CPS proposed by Naudet et al. (2023), providing practical examples of semantic integration in SoS environments.

5.1.4. The tooling ecosystem

Our analysis of the tooling ecosystem (RQ₃) reveals both areas of maturity and gaps that may impede the practical implementation of runtime composition in SoS.

Simulation-implementation gap. While our results show rich ecosystems in both simulation tools (e.g., Mosaik, Gazebo) and implementation frameworks (e.g., EAF, Docker), the fragmentation of these tools (most operating in isolation) means there is a notable lack of integrated toolchains bridging the simulation and implementation phases (with NVIDIA Omniverse in S68 as a rare exception). This disconnect between simulation-validated approaches and practical deployments may explain why many promising solutions fail to transition to practice.

The eclipse ecosystem. This ecosystem emerged as a significant contributor, highlighting the demand for cohesive, extensible solutions and representing a positive trend toward greater interoperability and knowledge sharing in the field.

5.1.5. Evaluation methods

Our analysis of evaluation methods (RQ₄) reveals several patterns that have implications for assessing the effectiveness of the proposed solutions. This analysis aligns with Santos et al. (2022), who focused on evaluating SoS architectures. Our study builds on this by broadly examining SoSE strategies, including a detailed taxonomy of evaluation methods and application domains.

Simulation dominance: Strength or limitation. The dominance of simulation-based evaluation methods, particularly scenario-based simulations, aligns with findings from de Amorim and Braga (2020), who investigated agent-based modeling to simulate SoS. Our analysis extends their work by identifying a broader range of simulation approaches beyond agent-based modeling and critically examining the limitations of simulation-centric evaluation in runtime composition. However, this reliance on simulation may inadvertently bias research toward solutions that perform well in simulated environments but may not address the full complexity of real-world SoS challenges with emergent behaviors and unanticipated interactions.

Domain diversity: Strength in breadth, weakness in depth. The diversity of application domains identified in our analysis—from industrial automation to emergency response—demonstrates the broad relevance of runtime composition across different SoS contexts. However, this diversity also results in fragmented evaluation methods that make comparing solutions across domains or developing domain-specific best practices difficult.

This fragmentation is particularly evident in the literature's limited representation of specific SoS types—directed and collaborative. Most studies (16) refer to general SoS without specifying a particular kind, suggesting insufficient attention to how different SoS contexts might require different runtime composition approaches. The absence of these SoS types suggests an opportunity for future research to address them explicitly.

This fragmentation is underscored by the limited attention given to specific SoS types, particularly directed and collaborative systems, which are not explicitly addressed in reviewed studies. Most studies refer only to general SoS without specifying a type, suggesting a lack of consideration for how varying SoS contexts may require distinct runtime composition strategies. This fragmentation highlights a promising direction for future research to investigate and characterize underrepresented SoS types more explicitly.

Insufficient attention to scalability. Many studies demonstrate solutions in controlled environments with limited CSs, raising questions about scalability to large-scale SoS with hundreds or thousands of CSs. This limitation is particularly concerning given the growing scale and complexity of real-world SoS deployments.

5.2. Research gaps and future directions

Based on our analysis of results, we identify the following research gaps and directions for future work.

5.2.1. Bridging technical and socio-technical dimensions

While the literature provides rich coverage of technical challenges and solutions for runtime composition, there is insufficient attention to the socio-technical dimensions that often determine practical success. This gap aligns with findings from Maciel et al. (2024), who identified a significant shift in interoperability research from purely technical concerns to socio-technical issues. Their tertiary study highlighted that interoperability now extends beyond software systems and requires multidisciplinary approaches. This need is echoed by the organizational and human factor gaps identified in RQ₁, which our findings show are still lacking in SoS contexts.

Future research should develop frameworks that explicitly address how organizational, human, commercial, and regulatory factors interact with technical solutions in SoS environments. Particularly promising are approaches that combine technical standards with governance frameworks, incentive mechanisms, and business models that encourage interoperability across organizational boundaries. Such holistic approaches are essential for addressing the proprietary constraints and organizational coupling identified in our challenge taxonomy.

5.2.2. AI-enabled adaptation and orchestration

The limited penetration of AI and machine learning (ML) techniques in current solutions represents future research opportunities. AI approaches could address several challenges identified in our taxonomy, including emergence management, dynamic discovery, and adaptive control strategies.

This gap aligns with findings from Nilsson et al. (2024), highlighting the underrepresentation of ML techniques in SoS research. Our analysis confirms this gap while providing specific examples of where AI/ML could be most effectively applied in runtime composition contexts.

Particularly promising are RL approaches for developing adaptation policies in uncertain environments (S73) and graph neural networks for inferring resilience strategies (S72). These approaches can address challenges identified by Hezavehi et al. (2021) regarding uncertainty in self-adaptive systems, particularly in managing unanticipated changes. Future research should explore how these techniques can be integrated into comprehensive frameworks that address the *autonomy-coordination paradox* while maintaining safety and predictability.

5.2.3. The benchmarking problem

A methodological gap emerges from our analysis of evaluation methods. Current research predominantly validates solutions through isolated simulations or proof-of-concept implementations based on scenario-specific assumptions (Table 9). This fragmentation creates two fundamental problems: evaluation metrics remain inconsistently defined across studies, and proposed solutions are rarely benchmarked against competing approaches.

While these isolated evaluations provide valuable insights into individual solutions, they prevent rigorous comparative assessment. This is unlike the ML domain, where standardized protocols, benchmarking datasets, and unified metrics accelerate progress by requiring new methods to demonstrate measurable improvements over existing baselines, fostering reproducibility and cumulative advancement.

The absence of standard benchmarks in SoS runtime composition research impedes cross-approach comparison and slows innovation momentum. Most studies compare against only self-developed baselines, except a few utilize common reference architectures. This absence undermines claims of generalizability and hinders progress, particularly for challenges requiring cross-organizational collaboration.

Future research should prioritize developing benchmarks that reflect real-world SoS constraints, including (1) standardized metrics that capture both technical performance and socio-technical factors and (2) shared platforms supporting cross-domain evaluations. Such standardization would facilitate more robust evaluation and foster the collaboration and incremental improvement that have driven rapid advancement in other technical fields.

5.3. Practitioner recommendations

Our findings have several implications for practitioners working on SoS development and integration.

5.3.1. Architectural choices: Service-orientation as foundation

The convergence around service-oriented architectural approaches, particularly the EAF, suggests that practitioners consider these paradigms foundational for new SoS developments. The widespread adoption across multiple studies indicates a level of maturity and community support that can reduce implementation risks.

This recommendation aligns with the catalog of design patterns presented by Axelsson (2022), who identified service-oriented approaches as one of the key patterns for SoS. Our findings extend this work by providing specific evidence of successful implementations and highlighting the EAF as a promising standardization effort.

However, practitioners should know the limitations of purely service-oriented approaches, particularly for systems with significant

physical components or real-time constraints. In such cases, complementary approaches, e.g., digital twins (S40, S61), may be necessary to address the complete integration challenges. This consideration is essential in light of findings from Olsson and Axelsson (2023) regarding the challenges of digital twins in SoS contexts, highlighting the need for a deeper understanding of their concepts, principles, costs, and benefits.

5.3.2. Tool selection strategy: Ecosystem over individual tools

Practitioners should prioritize coherent tool ecosystems over individual tool capabilities when selecting technology. The prominence of integrated ecosystems like Eclipse in the literature indicates the importance of tool interoperability and community support for successful SoS implementation.

When selecting tools, practitioners should evaluate current capabilities and ecosystem vitality, standardization efforts, and integration with complementary tools spanning the entire SoS lifecycle—from design to deployment and operation. This ecosystem-focused approach can help avoid the fragmentation challenges identified in our analysis and should be complemented by consideration of organizational and human factors that influence successful adoption.

6. Threats to validity

SLRs inevitably face certain limitations and risks that may impact the trustworthiness of their findings. This section outlines the key threats to validity inherent to the study's context and design, and describes the strategies employed to mitigate their influence. While many threats were addressed through rigorous methods, some residual limitations remain unavoidable and are transparently acknowledged to support critical appraisal and future replication efforts.

6.1. External validity

External validity—the extent to which our findings can be generalized beyond the specific studies and contexts included—can be jeopardized if the search process misses relevant work. To mitigate this risk, we

1. performed comprehensive searches across six leading digital libraries (chosen for their broad coverage, quality content, and prevalent use in SLR practice),
2. constructed a robust search string incorporating multiple keyword combinations drawn from established SoS terminology (Section 3.1.3), and
3. complemented the automated search with backward and forward snowballing.

Consistent with Wohlin's guidelines (Wohlin, 2014), snowballing was performed using the 77 primary studies retained after title/abstract screening, full-text screening, and quality assessment, ensuring a reliable start set. This avoided the infeasibility of applying snowballing to thousands of raw search hits while still maximizing coverage. Following Kitchenham et al. (2015), the same inclusion and exclusion criteria were systematically applied, and candidate studies meeting these criteria were added to the corpus. While primarily a safeguard for external validity, snowballing also contributes to internal validity (Section 6.2) by reducing the risk of omitting relevant studies due to search string limitations. This two-pronged approach increases confidence in the completeness of our dataset.

We acknowledge, however, that the five-year publication window (2019–2024) may have excluded earlier but still relevant foundational studies. This constraint was necessary to ensure the feasibility of the review, given the high volume of recent publications in the field. As Kitchenham et al. (2015) note, such scoping decisions are acceptable when they make in-depth synthesis manageable. Future reviews could expand the temporal scope to explore longitudinal trends.

Finally, including low-quality or tangentially relevant work can bias conclusions. We applied the established SLR quality-assessment checklist to every candidate study, filtering out those failing to meet our pre-defined thresholds of methodological rigor and venue reputation. Only studies that satisfied all quality criteria were retained.

6.2. Internal validity

Internal validity refers to the soundness and consistency of our thematic classifications and interpretations for each RQ. A_1 and A_3 independently drafted initial classification schemes in parallel to generate candidate taxonomies. The A_1 then consolidated these drafts into a single taxonomy, refining definitions and resolving discrepancies through full-text review and iterative discussion.

While we did not perform a formal inter-rater reliability statistic, this collaborative, back-and-forth refinement process ensured careful cross-validation of themes. We acknowledge that a single-author finalization step limits our ability to quantify agreement. Future work could include double-coding a subset of studies to quantify reliability and further strengthen the coding process.

6.3. Construct validity

Construct validity concerns whether the operational measures in this SLR truly capture the concepts we intend to study. Key threats include

- (a) selection bias in how we defined and applied inclusion/exclusion criteria,
- (b) instrumentation bias in data extraction, and
- (c) ambiguity in how key terms—such as runtime composition in SoS contexts—are interpreted across research communities. To mitigate these threats, we:
 - Defined core concepts (e.g., runtime composition, interoperability, adaptation, dynamic reconfiguration) based on foundational literature, before data extraction.
 - Developed a standardized codebook with clear definitions for each data item (D_1 – D_{10}), ensuring that constructs such as “challenge themes” and “tool categories” were consistently interpreted across studies.
 - Pilot-tested our data-extraction protocol on a random sample of 10 primary studies, refining ambiguous codes before complete extraction.
 - Employed dual independent extraction: two reviewers extracted data in parallel, then reconciled discrepancies through discussion.

6.4. Conclusion validity

Conclusion validity pertains to the reliability of inferences drawn from the data. Potential threats include (1) researcher bias in TA, and (2) over-generalization of findings. We mitigated these by:

- Using a structured data extraction form to gather consistent information across all studies.
- Maintained detailed records of our decision-making processes throughout the analysis.
- Applied established qualitative analysis techniques (TA).
- Triangulated findings across multiple sources (e.g., cross-referencing identified challenges with proposed solutions).
- Provided extensive traceability between our findings and the primary studies through comprehensive referencing.

7. Related work

This section examines prior systematic reviews, mapping studies, and surveys related to this study. Table 10 provides a detailed comparison

between our SLR and existing reviews across key dimensions, highlighting our distinct contributions.

Our SLR builds upon and extends existing literature in several important ways. While previous reviews have focused on specific aspects of SoSE, our work offers a unified taxonomy that integrates multiple dimensions, including runtime composition challenges, tools, evaluation methods, and domain applications.

7.1. Architecture and modeling approaches

Previous architectural studies have explored various patterns and frameworks for SoS design. Garcés et al. (2021) proposed the HSH-SoS framework, categorizing major architectural styles (Centralized, SOA, MSA), while Fang (2022) analyzed factors influencing architecture selection under dynamic evolution. In the modeling domain, Eichmann et al. (2019) classified techniques using UAF and V-Model frameworks for handling dynamic changes.

While valuable, these architectural and modeling studies typically address isolated aspects of SoSE. Our taxonomy situates these architectural concerns within a broader classification that organizes runtime composition methods and their evaluation methods by thematic category, linking architectural choices to specific challenges and solutions.

7.2. Evaluation and interoperability

Several reviews have focused on evaluation methods or interoperability challenges. Santos et al. (2022) cataloged various methods and metrics for evaluating SoS architectures, reinforcing our finding on the dominance of simulation in such evaluations. Interoperability studies by Maciel et al. (2024) identified diverse types and socio-technical concerns, while Nilsson et al. (2024) and Naudet et al. (2023) explored AI methods and cognitive aspects of interoperability, respectively.

Our SLR situates these evaluation and interoperability concerns within a comprehensive mapping of solution approaches for runtime composition, providing context for when and how different methods apply across the SoS lifecycle.

7.3. Emerging approaches and domain applications

Hezavehi et al. (2021) explored approaches for managing unanticipated changes in self-adaptive systems, Olsson and Axelsson (2023) analyzed digital twins as an integration approach, and Fortino et al. (2021) investigated IoT-SoS integration. Axelsson (2022) synthesized a catalog of SoS design patterns, confirming SOA's importance.

Our work provides a domain-agnostic analysis of emerging approaches to runtime composition and adaptation in SoS, integrating insights from diverse studies to offer a comprehensive taxonomy that maps challenges, solutions, tools, and evaluation methods across multiple dimensions, with particular emphasis on runtime composition—an aspect that cuts across prior review boundaries.

8. Conclusion

This SLR examined the landscape of runtime composition in dynamic SoSs, analyzing 80 studies published between 2019 and 2024. Our goal was to synthesize current knowledge on challenges, solution approaches, tools, and evaluation methods to provide a coherent understanding of runtime-adaptive SoSs.

We identified four primary challenge themes—Modeling & Analysis, Robust & Resilient Operations, System Orchestration, and Heterogeneity of CSs—comprising over 50 specific technical obstacles, including emergent behavior prediction, workflow indeterminacy, and black-box component integration. Proposed solutions clustered into seven thematic areas: co-simulation & digital twins, semantic ontologies, integration frameworks, architectural languages & SOA, middleware &

Table 10

Comparison of results between our SLR and existing secondary studies.

Study	Scope	Key Findings	Overlap	Distinct Contributions
(Garcés et al., 2021)	Architectural patterns in SoS (Centralized, SOA, MSA).	Pattern-based framework (HSH-SoS) categorizing major SoS design styles.	Confirms prevalence of SOA/MSA patterns.	We extend beyond patterns to runtime-composition challenges and tools.
(Fang, 2022)	Factors influencing architecture selection under dynamic evolution.	Emphasis on interdependency, autonomy, and uncertainty.	Agrees that dynamic evolution demands adaptive architectures.	We map specific evaluation methods across themes.
(Eichmann et al., 2019)	Model-based frameworks (UAF + V-Model) for handling dynamic changes.	Classification of modeling techniques for SoS development.	Shares interest in modeling and V&V.	We offer a unified taxonomy integrating modeling with resilience, tools, and analytics.
(Santos et al., 2022)	Evaluation methods and metrics for SoS architectures.	Cataloged evaluation methods, metrics, and trade-offs in SoS architecture assessment.	Reinforces our finding on simulation dominance in evaluation.	We embed evaluation within a broader SoSE framework (incl. tools, domains).
(Maciel et al., 2024)	Interoperability types in systems.	Identified diverse interoperability types and shift toward socio-technical concerns	Interoperability challenges in complex systems.	We incorporate interoperability within a broader taxonomy of SoS challenges and map specific solutions for runtime composition.
(Nilsson et al., 2024)	AI methods for dynamic interoperability.	Covered AI approaches like ontology alignment, heterogeneous data inference, and neural transcoding.	Underrepresentation of AI-driven approaches for interoperability.	We situate AI approaches within a comprehensive mapping of all solution types for runtime composition in SoS.
(Naudet et al., 2023)	Cognitive interoperability in CPS enterprises.	Identified gaps in maturity models and the need for human-machine collaboration.	Cognitive aspects of system integration	We map cognitive interoperability within the broader context of runtime composition challenges in SoS.
(Hezavehi et al., 2021)	Uncertainty in self-adaptive systems.	Explored community perceptions and approaches for managing unanticipated changes.	Explored uncertainty challenges	We integrated these challenges in the broader taxonomy.
(Olsson and Axelsson, 2023)	Digital twins in SoS.	Found certain digital twin challenges that need further study.	Confirms digital twins as an SoS integration approach.	We analyze digital twins as one of several solution approaches within a comprehensive framework for runtime composition.
(Axelsson, 2022)	Design patterns in SoS	Synthesized a catalog of SoS design patterns.	Confirms SOA as a key design pattern for SoS.	Our SLR examines patterns alongside other solutions to enable runtime composition and adaptation.
(Fortino et al., 2021)	IoT in SoS contexts.	Analyzed methodologies, frameworks, and tools for IoT-SoS integration	Confirms IoT as one of the key domains for modern dynamic SoSs.	We provide a domain-agnostic analysis of runtime composition challenges and solutions.

communication, formal & analytical methods, and resilience & AI-driven approaches.

Our analysis highlights that interoperability, heterogeneity, and the autonomy-coordination paradox remain central challenges, while SOA, semantic technologies, and adaptive frameworks dominate proposed solutions. Tooling is concentrated around platforms such as the EAF and simulation environments like Mosaik and Gazebo, yet fragmentation persists between design-time models and runtime dynamics.

These findings underscore the importance of holistic approaches that integrate technical and socio-technical dimensions, including governance frameworks and AI-driven adaptation strategies. Notably, gaps remain in benchmarking, scalability validation, and explicit support for directed and collaborative SoS types.

By providing a structured mapping of challenges, solutions, tools, and evaluation methods, this review offers researchers and practitioners

a clear picture of the current state of runtime composable SoSs. Future research should focus on standardized evaluation metrics, integrated toolchains, and interdisciplinary collaboration to advance runtime-adaptive SoSs from theory to practice.

CRediT authorship contribution statement

Muhammad Ashfaq: Conceptualization, Methodology, Software; **Ahmed R. Sadik:** Data curation, Writing – original draft; **Teerath Das:** Data curation, Writing – original draft; **Muhammad Waseem:** Data curation, Writing – original draft; **Niko Mäkitalo:** Data curation, Writing – Original draft preparation; **Tommi Mikkonen:** Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We thank Dr. Rahul Mohanani and Dr. Nauman bin Ali for their helpful suggestions in improving the quality of this study.

Appendix A. Acronyms

See [Table A.1](#).

Table A.1

List of acronyms.

Acronym	Full Form
ADL	Architecture Description Language
API	Application Programming Language
BERT	Bidirectional Encoder Representations from Transformers
CPS	Cyber-Physical System
CS(s)	Constituent System(s)
CSoS	Combat SoS
CSSAD	CSoS autonomous decision-making
DAB	Discovery Access Broker
DCS	Discrete Controller Synthesis
DoSo	Degree of System Order
DSL	Domain-Specific Language
EAF	Eclipse Arrowhead Framework
IIoT	Industrial IoT
IoT	Internet of Things
ISoS	Infrastructure SoS
JSON	JavaScript Object Notation
MAPE-K	Monitor-Analyze-Plan-Execute over Knowledge
MPLS	Multiprotocol Label Switch
MQTT	Message Queuing Telemetry Transport
MSA	Micro-Service Architecture(s)
O/M independence	Operational and Managerial Independence
OPC UA	Open Platform Communications - Unified Architecture
OS	Operating System(s)
RDF	Resource Description Format
ReViTA	Reconfigurations Via Transient Architectural Configurations
RQ(s)	Research Question(s)
SiSoS	Software-intensive SoS
SLR	Systematic Literature Review
SOA	Service-Oriented Architecture(s)
SoS(s)	System(s) of Systems
SoSE	System of Systems Engineering
TA	Thematic Analysis
TSN	Time-Sensitive Networking

Appendix B. Selected studies

See [Table B.1](#).

Table B.1

List of selected studies for the SLR.

ID	Study Title	Citation	Score
S01	Applying model-based co-simulation on modular production units in complex automation systems.	(Binder et al., 2021)	4
S02	Engineering interoperable, plug-and-play, distributed, robotic control systems for futureproof fusion power plants.	(Caliskanelli et al., 2021b)	4
S03	Machine learning-based discrete event dynamic surrogate model of communication systems for simulating the command, control, and communication system of systems.	(Kang et al., 2019)	4
S04	Modeling challenges for Earth observing systems of systems	(Grogan, 2019)	5
S05	An interoperable tool-chain for energy monitoring applications.	(Brunelli et al., 2019)	4
S06	MicroGraphQL: A unified communication approach for systems of systems using microservices and GraphQL.	(De F. Borges et al., 2022)	5
S07	Towards an OPC UA Compliant Programming Approach with Formal Model of Computation for Dynamic Reconfigurable Automation Systems.	(Atmojo and Vyatkin, 2019)	3
S08	Autonomous and composable M&S system of systems with the simulation, experimentation, analytics and testing (SEAT) framework.	(Mittal et al., 2020)	4
S09	Building Systems of Systems with Escher.	(Canakci et al., 2021)	4
S10	Towards sustainable systems reconfiguration by an IoT-driven system of systems engineering lifecycle approach.	(Forte et al., 2022)	3
S11	A resilience implementation framework of system-of-systems based on hypergraph model.	(Jiang et al., 2022)	3
S12	Native OPC UA handling and IEC 61,499 PLC integration within the arrowhead framework.	(Cabral et al., 2020)	4
S13	Efficiently Composing Validated Systems Integration Gateways for Dynamic, Diverse Data.	(Rock et al., 2019)	3
S14	Multi-scale hydrological system-of-systems realized through WHOS: the brokering framework.	(Boldrini et al., 2022)	3
S15	A framework for the design of fault-tolerant systems-of-systems.	(Ferreira et al., 2024)	3
S16	Adaptive Workflow of Service Oriented IoT Architectures for Small and Distributed Automation Systems.	(Zabasta et al., 2020)	4
S17	Experiences of using linked data and ontologies for operational data sharing in systems-of-systems.	(Axelsson, 2019)	5
S18	Enabling architecture based co-simulation of complex Smart Grid applications.	(Binder et al., 2019)	5
S19	Evolving the ecosystem: Eclipse arrowhead integrates eclipse IoT.	(Kristan et al., 2022)	4
S20	Platooning LEGOs: An open physical exemplar for engineering self-adaptive cyber-physical systems-of-systems.	(Shin et al., 2021)	4
S21	From models to management and back: Towards a system-of-systems engineering toolchain.	(Kulcsár et al., 2020)	3.5
S22	Fuzzy architecture description for handling uncertainty in IoT systems-of-systems.	(Oquendo, 2020)	4
S23	IPSME-Idempotent Publish/Subscribe Messaging Environment.	(Nevelsteen and Wehlou, 2021)	3
S24	System of systems lifecycle management—A new concept based on process engineering methodologies	(Kozma et al., 2021)	3
S25	Improving system-of-systems agility through dynamic reconfiguration.	(Fang et al., 2020)	3.5
S26	Cautious adaptation of defiant components.	(Maia et al., 2019)	3
S27	An ontological framework for opportunistic composition of IoT systems.	(Nundloll et al., 2020)	5
S28	Research on Autonomous Reconstruction Method for Dependent Combat Networks.	(Sun and Zhang, 2023)	3
S29	A games-in-games approach to mosaic command and control design of dynamic network-of-networks for secure and resilient multi-domain operations.	(Chen and Zhu, 2019)	4
S30	Software mediators as first-class entities of systems-of-systems software architectures.	(Garcés et al., 2019)	5
S31	Conceptual, Mathematical, and Analytical Foundations for Mission Engineering and System of Systems Analysis.	(Raz et al., 2024)	3.5
S32	NLP-based generation of ontological system descriptions for composition of smart home devices.	(Zhang et al., 2023)	3
S33	Continuous verification with acknowledged MAPE-K pattern and time logic-based slicing: A platooning system of systems case study.	(Song and Bae, 2023)	3.5
S34	Integrating systems of systems with a federation of rule engines.	(Berezovskyi et al., 2024)	3.5
S35	Incremental, distributed, and concurrent service coordination for reliable and deterministic systems-of-systems.	(Dhiah el Diehn et al., 2020)	3
S36	Providing a User Extensible Service-Enabled Multi-Fidelity Hybrid Cloud-Deployable SoS Test and Evaluation (T&E) Infrastructure: Application of Modeling and Simulation (M&S) as a Service (MSaaS).	(Mittal et al., 2023)	3.5
S37	Design process for system of systems reconfigurations.	(Petitdémange et al., 2021)	4
S38	Adore: An adaptation-oriented requirement modeling approach for systems of systems.	(Maciel et al., 2019)	5
S39	Heterogeneous Network-Driven Data Fabrics to Enable Multi-Mission Autonomy.	(Hawkins et al., 2022)	3.5
S40	Digital twins for collaboration and self-integration.	(Esterle et al., 2021)	4
S41	Continuous verification of system of systems with collaborative MAPE-K pattern and probability model slicing.	(Song et al., 2022)	3.5
S42	Modeling the dynamic reconfiguration in smart crisis response systems.	(Seghiri et al., 2022)	4
S43	HolonCraft—an architecture for dynamic construction of smart home workflows.	(Wang et al., 2022)	4.5
S44	Simulation-Based Engineering of Heterogeneous Collaborative Systems—A Novel Conceptual Framework.	(Perišić et al., 2023)	4
S45	Extending the CST: The Distributed Cognitive Toolkit.	(Gibaut and Gudwin, 2020)	3.5
S46	Principled and automated system of systems composition using an ontological architecture.	(Elhabbash et al., 2024)	4.5
S47	Simulation integration platforms for cyber-physical systems.	(Neema et al., 2019)	5
S48	A cloud-based development environment using HLA and kubernetes for the co-simulation of a corporate electric vehicle fleet.	(Rehman et al., 2019)	4
S49	Interoperability analysis via agent-based simulation.	(Pescatore and Beery, 2024)	3.5
S50	A Maude-Based Formal Approach to Control and Analyze Time-Resource Aware Missioned Systems-of-Systems.	(Eddine et al., 2023)	3.5
S51	Self-Adaptation of Loosely Coupled Systems across a System of Small Uncrewed Aerial Systems.	(Chambers et al., 2024)	4
S52	System of system composition based on decentralized service-oriented architecture.	(Derhamy et al., 2019)	5
S53	Mission reliability modeling and evaluation for reconfigurable unmanned weapon system-of-systems based on effective operation loop.	(Chen et al., 2023)	3
S54	A middleware for automatic composition and mediation in IoT systems.	(Elhabbash et al., 2022)	4.5
S55	Communication oriented modeling of evolving systems of systems.	(Harbo et al., 2021)	3.5
S56	The interoperability challenge: building a model-driven digital thread platform for CPS.	(Margaria et al., 2021)	4
S57	Modeling & simulation of software architectures of systems-of-systems: An industrial report on the Brazilian space system.	(Neto et al., 2019)	5
S58	Two-way integration of service-oriented systems-of-systems with the web of things.	(Zyrianoff et al., 2021)	4
S59	Hardware/software self-adaptation in CPS: the CERBERO project approach.	(Palumbo et al., 2019)	4
S60	JARVIS, A Hardware/Software Framework for Resilient Industry 4.0 Systems.	(Parri et al., 2019)	3
S61	Service-oriented reconfiguration in systems of systems assured by dynamic modular safety cases.	(Thomas et al., 2021)	3
S62	On Combining Domain Modeling and Organizational Modeling for Developing Adaptive Cyber-Physical Systems.	(Sudeikat and Köhler-Bußmeier, 2022)	3
S63	Interoperability for industrial internet of things based on service-oriented architecture.	(Lam et al., 2021)	4
S64	System of System Strategy for Multi-Level Interoperability for Smart Cities.	(Loss et al., 2023)	5
S65	CorteX: A software framework for interoperable, plug-and-play, distributed, robotic systems of systems.	(Caliskanelli et al., 2021a)	4

Table B.1
Continued

ID	Study Title	Citation	Score
S66	Employing discrete controller synthesis for developing systems-of-systems controllers.	(Li et al., 2024)	4
S67	A Maude-Based Rewriting Approach to Model and Control System-of-Systems' Resources Allocation.	(Dridi et al., 2022)	3.5
S68	Systems of Digital Twins and Physical Systems: Interoperability, Decentralization, and Mobility in Robotic Applications.	(Girke et al., 2024)	5
S69	DynADL-Dynamic Architecture Description Language for System of Systems.	(Hristozov and Matson, 2024)	5
S70	A system of systems architecture for the internet of things exploiting autonomous components.	(Nikolopoulos et al., 2019)	4
S71	Multi-swarm-based cooperative reconfiguration model for resilient unmanned weapon system-of-systems.	(Sun et al., 2022)	3
S72	Deep reinforcement learning-based resilience enhancement strategy of unmanned weapon system-of-systems under inevitable interferences.	(Sun et al., 2024)	3
S73	Generation of Adaptation Strategies for Dynamic Reconfiguration of a System of Systems.	(Lee et al., 2021)	4
S74	System reconfiguration ontology to support model-based systems engineering: Approach linking design and operations.	(Qasim et al., 2023)	4
S75	A Brief Introduction into (De-)Coupling Lifecycles in Net-Centric Systems-of-Systems.	(Schütz and Marx Gómez, 2022)	4
S76	KIPO opportunities for interoperability decisions in systems-of-information systems in the domain of environmental management.	(Fernandes et al., 2017)	4
S77	Exploring system of systems resilience versus affordability trade-space using a bio-inspired metric.	(Chatterjee et al., 2021)	3
S78	Using modeling and simulation and artificial intelligence to improve complex adaptive systems engineering.	(Tolk et al., 2022)	3
S79	Architecture Description Method for Open Systems-of-Systems to Reduce Misunderstanding the Scopes of Managed Objects.	(Kobayashi et al., 2020)	4
S80	A self-adaptive system of systems architecture to enable its ad-hoc scalability: Unmanned vehicle fleet-mission control center case study.	(Sadik et al., 2023)	4

References

- Agarwal, P., Estanguet, R., Leblanc, F., Ernadote, D., 2023. Architecting system of systems for resilience using MBSE. In: IEEE 18th Annual System of Systems Engineering Conference, pp. 1–7. <https://doi.org/10.1109/SOSE59841.2023.10178534>
- de Amorim, S.R., Braga, R. T.V., 2020. Simulating systems-of-systems with agent-based modeling: a systematic literature review. *IEEE Syst. J.* 14 (3), 3609–3617. <https://doi.org/10.1109/JSYST.2020.2980896>
- Ashfaq, M., Sadik, A.R., Mikkonen, T., Waseem, M., Mäkitalo, N., 2024. Holon programming model: a software-defined approach for system of systems. In: 11th International Conference on Software Defined Systems, pp. 104–111. <https://doi.org/10.1109/SDS64317.2024.10883886>
- Atmojo, U.D., Vyatkin, V., 2019. Towards an OPC UA compliant programming approach with formal model of computation for dynamic reconfigurable automation systems. In: 17th International Conference on Industrial Informatics. Vol. 1, pp. 140–146. <https://doi.org/10.1109/INDIN41052.2019.8972084>
- Axelsson, J., 2019. Experiences of using linked data and ontologies for operational data sharing in systems-of-systems. In: IEEE International Systems Conference, pp. 1–8. <https://doi.org/10.1109/SYSCON.2019.8836909>
- Axelsson, J., 2022. Systems-of-systems design patterns: a systematic literature review and synthesis. In: 17th Annual System of Systems Engineering Conference, pp. 171–176. <https://doi.org/10.1109/SOSE55472.2022.9812681>
- Benabidallah, R., Sadou, S., Esnault, A., Nacer, M.A., 2020. Simulating systems of systems using situation/reaction paradigm. *Concurr. Comput. Pract. Exp.* 32 (15), e4921. <https://doi.org/10.1002/cpe.4921>
- Berezovskyi, A., Inam, R., El-khoury, J., Mokrushin, L., Fersman, E., 2024. Integrating systems of systems with a federation of rule engines. *J. Indust. Inf. Integr.* 38, 100545. <https://doi.org/10.1016/J.JII.2023.100545>
- Binder, C., Agic, A., Neureiter, C., Lüder, A., 2021. Applying model-based co-simulation on modular production units in complex automation systems. In: 2021 IEEE International Symposium on Systems Engineering, pp. 1–6. <https://doi.org/10.1109/ISSE51541.2021.9582531>
- Binder, C., Fischinger, M., Altenhuber, L., Draxler, D., Lastro, G., Neureiter, C., 2019. Enabling architecture based co-simulation of complex smart grid applications. *Energy Inf.* 2, 1–19. <https://doi.org/10.1186/S42162-019-0084-0>
- Blair, G., Bromberg, Y.-D., Coulson, G., Elkhatib, Y., Réveillère, L., Ribeiro, H.B., Rivière, E., Taïani, F., 2015. Holons: towards a systematic approach to composing systems of systems. In: 14th International Workshop on Adaptive and Reflective Middleware, pp. 1–6. <https://doi.org/10.1145/2834965.2834970>
- Boldrini, E., Nativi, S., Pecora, S., Chernov, I., Mazzetti, P., 2022. Multi-scale hydrological system-of-systems realized through WHOS: the brokering framework. *Int. J. Digital Earth* 15 (1), 1259–1289. <https://doi.org/10.1080/17538947.2022.2099591>
- Braun, V., Clarke, V., 2006. Using thematic analysis in psychology. *Qual. Res. Psychol.* 3 (2), 77–101. <https://doi.org/10.1191/1478088706qp0630a>
- Braun, V., Clarke, V., 2019. Reflecting on reflexive thematic analysis. *Qualit. Res. Sport, Exer. Health* 11 (4), 589–597. <https://doi.org/10.1080/2159676X.2019.1628806>
- Brunelli, D., Cinotti, T.S., Woehrl, H., Aguzzi, C., Montori, F., Benini, L., 2019. An interoperable tool-chain for energy monitoring applications. In: 2019 AEIT International Annual Conference, pp. 1–6. <https://doi.org/10.23919/AEIT.2019.8893354>
- Cabral, J., Dorofeev, K., Varga, P., 2020. Native OPC UA handling and IEC 61499 PLC integration within the arrowhead framework. In: IEEE Conference on Industrial Cyberphysical Systems. Vol. 1, pp. 596–601. <https://doi.org/10.1109/ICPS48405.2020.9274768>
- Caliskanelli, I., Goodliffe, M., Whiffin, C., Xymitoulas, M., Whittaker, E., Verma, S., Hickman, C., Minghao, C., Skilton, R., 2021a. CorteX: A Software Framework for Interoperable, Plug-and-Play, Distributed, Robotic Systems of Systems. Springer. chapter 10, pp. 295–344. https://doi.org/10.1007/978-3-030-66494-7_10
- Caliskanelli, I., Goodliffe, M., Whiffin, C., Xymitoulas, M., Whittaker, E., Verma, S., Skilton, R., 2021b. Engineering interoperable, plug-and-play, distributed, robotic control systems for futureproof fusion power plants. *Robotics* 10 (3), 108. <https://doi.org/10.3390/robotics10030108>
- Canacki, B., Alvisi, L., van Renesse, R., 2021. Building systems of systems with Escher. In: International Symposium on Stabilizing, Safety, and Security of Distributed Systems, pp. 34–50. https://doi.org/10.1007/978-3-030-91081-5_3
- Chambers, T., Cleland-Huang, J., Vierhauser, M., 2024. Self-adaptation of loosely coupled systems across a system of small uncrewed aerial systems. In: 12th ACM/IEEE International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems, pp. 37–44. <https://doi.org/10.1145/3643655.3643882>
- Chatterjee, A., Malak, R., Layton, A., 2021. Exploring system of systems resilience versus affordability trade-space using a bio-inspired metric. *J. Comput. Inf. Sci. Eng.* 21 (5), 050905. <https://doi.org/10.1115/1.4050288>
- Chen, J., Zhu, Q., 2019. A games-in-games approach to mosaic command and control design of dynamic network-of-networks for secure and resilient multi-domain operations. In: Sensors and Systems for Space Applications XII. Vol. 11017, pp. 189–195. <https://doi.org/10.1117/12.2526677>
- Chen, Z., Zhou, Z., Zhang, L., Cui, C., Zhong, J., 2023. Mission reliability modeling and evaluation for reconfigurable unmanned weapon system-of-systems based on effective operation loop. *J. Syst. Eng. Electron.* 34 (3), 588–597. <https://doi.org/10.23919/JSEE.2023.000082>
- Cruzes, D.S., Dyba, T., 2011. Recommended steps for thematic synthesis in software engineering. In: 2011 International Symposium on Empirical Software Engineering and Measurement. IEEE, pp. 275–284. <https://doi.org/10.1109/ESEM.2011.36>
- De F. Borges, M.V., Rocha, L.S., Maia, P. H.M., 2022. MicroGraphQL: a unified communication approach for systems of systems using microservices and GraphQL. In: 10th IEEE/ACM International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems, pp. 33–40. <https://doi.org/10.1145/3528229.3529381>
- Derhamy, H., Eliasson, J., Delsing, J., 2019. System of system composition based on decentralized service-oriented architecture. *IEEE Syst. J.* 13 (4), 3675–3686. <https://doi.org/10.1109/JSYST.2019.2894649>
- Dhiah el Diehn, I., Alouneh, S., Obermaier, R., et al., 2020. Incremental, distributed, and concurrent service coordination for reliable and deterministic systems-of-systems. *IEEE Syst. J.* 15 (2), 2470–2481. <https://doi.org/10.1109/JSYST.2020.3020430>
- Dridi, C.E., Hameurlain, N., Belala, F., 2022. A maude-based rewriting approach to model and control system-of-systems' resources allocation. In: International Conference on Model and Data Engineering, pp. 207–221. https://doi.org/10.1007/978-3-031-23119-3_15
- Eddine, C., Hameurlain, N., Belala, F., 2023. A maude-based formal approach to control and analyze time-resource aware missioned systems-of-systems. In: 2023 IEEE International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises, pp. 1–6. <https://doi.org/10.1109/WETICE57085.2023.10477839>
- Eichmann, O.C., Melzer, S., God, R., 2019. Model-based development of a system of systems using unified architecture framework (UAF): a case study. In: 2019 IEEE International Systems Conference, pp. 1–8. <https://doi.org/10.1109/SYSCON.2019.8836749>
- Elhabbash, A., Elkhatib, Y., Bouloukakis, G., Salama, M., 2022. A middleware for automatic composition and mediation in IoT systems. In: 12th International Conference on the Internet of Things, pp. 127–134. <https://doi.org/10.1145/3567445.3567451>
- Elhabbash, A., Elkhatib, Y., Nundloll, V., Marco, V.S., Blair, G.S., 2024. Principled and automated system of systems composition using an ontological architecture. *Future Gen. Comput. Syst.* 157, 499–515. <https://doi.org/10.1016/J.FUTURE.2024.03.034>

- Esterle, L., Gomes, C., Frasheri, M., Ejersbo, H., Tomforde, S., Larsen, P.G., 2021. Digital twins for collaboration and self-integration. In: 2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion, pp. 172–177. <https://doi.org/10.1109/ACSCS-C52956.2021.00040>
- eVTOL, 2025. Making mobility in the skies more accessible for people: Honda eVTOL. https://global.honda/en/tech/Electric_Vertical_Take-Off_and_Landing_aircraft_eVTOL/. Accessed: 2025-08-13.
- Fang, Z., 2022. System-of-systems architecture selection: a survey of issues, methods, and opportunities. *IEEE Syst. J.* 16 (3), 4768–4779. <https://doi.org/10.1109/JSYST.2021.3119294>
- Fang, Z., Liao, J., Zhou, X., 2020. Improving system-of-systems agility through dynamic reconfiguration. In: 2020 IEEE International Conference on Systems, Man, and Cybernetics, pp. 4466–4472. <https://doi.org/10.1109/SMC42975.2020.9283236>
- FCR, 2025. Honda Technology Ready To Supply Frequency Containment Reserve For Power Grid Stabilization. <https://hondanews.eu/en/cars/media/press-releases/429858/honda-technology-ready-to-supply-frequency-containment-reserve-for-power-grid-stabilization>. Accessed: 2025-08-13.
- Fernandes, J., Baião, F., Santos, R. P.d., 2017. KIPO opportunities for interoperability decisions in systems-of-information systems in the domain of environmental management. In: Workshop of Human-Computer Interaction Aspects to the Social Web, pp. 21–41. https://doi.org/10.1007/978-3-030-46130-0_2
- Ferreira, F. H.C., Nakagawa, E.Y., Bertolino, A., Lonetti, F., de Oliveira Neves, V., dos Santos, R.P., 2024. A framework for the design of fault-tolerant systems-of-systems. *J. Syst. Softw.* 211, 112010. <https://doi.org/10.1016/j.jss.2024.112010>
- Fortie, S., Dickopf, T., Weber, S., Göbel, J.C., 2022. Towards sustainable systems reconfiguration by an IoT-driven system of systems engineering lifecycle approach. *Procedia CIRP* 105, 654–659. <https://doi.org/10.1016/j.procir.2022.02.109>
- Fortino, G., Savaglio, C., Spezzano, G., Zhou, M., 2021. Internet of things as system of systems: a review of methodologies, frameworks, platforms, and tools. *IEEE Trans. Syst. Man Cybernet. Syst.* 51 (1), 223–236. <https://doi.org/10.1109/TSMC.2020.3042898>
- Galaway, U.o., 2024. Scopus and ScienceDirect – a comparison. Accessed 2025-01-16. <https://libguides.library.universityofgalway.ie/Scopusguide/ScopusScienceDirect-comparison>
- Garcés, L., Bruno, S., Nakagawa, E.Y., 2021. Towards an architectural patterns language for systems-of-systems. In: 26th Conference on Pattern Languages of Programs. USA, pp. 1–24. <https://dl.acm.org/doi/10.5555/3492252.3492254>
- Garcés, L., Oquendo, F., Nakagawa, E.Y., 2019. Software mediators as first-class entities of systems-of-systems software architectures. *J. Braz. Comput. Soc.* 25, 1–23. <https://doi.org/10.1186/S13173-019-0089-3>
- Gibaut, W., Gudwin, R., 2020. Extending the CST: the distributed cognitive toolkit. In: 2020 International Conferences on Internet of Things and IEEE Green Computing and Communications and IEEE Cyber, Physical and Social Computing and IEEE Smart Data and IEEE Congress on Cybermatics, pp. 474–481. <https://doi.org/10.1109/ITHINGS-GREENCOM-CPSOC-SMARTDATA-CYBERMATIC50389.2020.00088>
- Girke, F., Hermann, R., Kaigom, E.G., 2024. Systems of digital twins and physical systems: interoperability, decentralization, and mobility in robotic applications. In: 28th International Conference on Methods and Models in Automation and Robotics, pp. 29–34. <https://doi.org/10.1109/MMAR62187.2024.10680793>
- Grogan, P.T., 2019. Modeling challenges for earth observing systems of systems. In: IEEE International Geoscience and Remote Sensing Symposium, pp. 5289–5292. <https://doi.org/10.1109/IGARSS.2019.8898636>
- Harbo, S. K.R., Kristensen, M.K., Voldby, E.P., Andersen, S.V., Petersen, F.C., Albano, M., 2021. Communication oriented modeling of evolving systems of systems. In: 2021 16th International Conference of System of Systems Engineering, pp. 88–94. <https://doi.org/10.1109/SOSE52739.2021.9497495>
- Hawkins, M., Wang, A.K., Chang, T., Rowe, D., Martin, B., Enoch, M., Anderson, G., Pandya, J., 2022. Heterogeneous network-driven data fabrics to enable multi-mission autonomy. In: 2022 IEEE Military Communications Conference, pp. 259–264. <https://doi.org/10.1109/MILCOM55135.2022.10017623>
- Hezavehi, S.M., Weyns, D., Avgeriou, P., Calinescu, R., Mirandola, R., Perez-Palacin, D., 2021. Uncertainty in self-adaptive systems: a research community perspective. *ACM Trans. Auton. Adapt. Syst.* 15 (4), 1–36. <https://doi.org/10.1145/3487921>
- Hristozov, A.D., Matson, E.T., 2024. DynADL-dynamic architecture description language for system of systems. In: IEEE International Systems Conference, pp. 1–8. <https://doi.org/10.1109/SYSCON61195.2024.10553612>
- IEEE, 2016. IEEE Standard for Fuzzy Markup Language. Technical Report IEEE Std 1855–2016. IEEE. <https://doi.org/10.1109/IEEESTD.2016.7479441>
- INCOSE, 2014. Systems engineering vision 2025! <https://www.incose.org/2021-redesign/news-detail/2014/06/23/systems-engineering-vision-2025/>. Accessed: 2025-04-20.
- ISO/IEC/IEEE, 2019. Systems and Software Engineering – Taxonomy of Systems of Systems. Standard 21841:2019. ISO/IEC/IEEE. <https://doi.org/10.1109/IEEESTD.2019.8766998>
- Jamshidi, M. (Ed.), 2008. Systems of Systems Engineering: Principles and Applications. CRC Press. 1st edition. <https://doi.org/10.1109/MILCOM47813.2019.9020714>
- Jiang, J., Lakhal, O., Merzouki, R., 2022. A resilience implementation framework of system-of-systems based on hypergraph model. In: 17th Annual System of Systems Engineering Conference, pp. 487–492. <https://doi.org/10.1109/SOSE55472.2022.9812688>
- Kang, B.G., Seo, K.-M., Kim, T.G., 2019. Machine learning-based discrete event dynamic surrogate model of communication systems for simulating the command, control, and communication system of systems. *Simulation* 95 (8), 673–691. <https://doi.org/10.1177/0037549718809890>
- Kitchenham, B.A., Budgen, D., Brereton, P., 2015. Evidence-based software engineering and systematic reviews. Chapman and Hall/CRC. 1st edition. <https://doi.org/10.1201/b19467>
- Kobayashi, N., Fujino, H., Shirasaka, S., 2020. Architecture description method for open systems-of-systems to reduce misunderstanding the scopes of managed objects. *Intern. J. Japan Assoc. Manage. Syst.* 12 (1), 43–50. <https://doi.org/10.14790/ijams.12.43>
- Kozma, D., Varga, P., Larrinaga, F., 2021. System of systems lifecycle management—a new concept based on process engineering methodologies. *Appl. Sci.* 11 (8), 3386. <https://doi.org/10.3390/app11083386>
- Kristan, J., Azzoni, P., Römer, L., Jeroschewski, S.E., Londero, E., 2022. Evolving the ecosystem: eclipse arrowhead integrates eclipse IoT. In: IEEE/IFIP Network Operations and Management Symposium, pp. 1–6. <https://doi.org/10.1109/NOMS54207.2022.9789922>
- Kulcsár, G., Koltai, K., Tanyi, S., Péceli, B., Horváth, Á., Micskei, Z., Varga, P., 2020. From models to management and back: towards a system-of-systems engineering toolchain. In: IEEE/IFIP Network Operations and Management Symposium, pp. 1–6. <https://doi.org/10.1109/NOMS47738.2020.9110310>
- Lam, A.N., Haugen, Ø., Delsing, J., 2021. Interoperability for industrial internet of things based on service-oriented architecture. In: 47th Annual Conference of the IEEE Industrial Electronics Society, pp. 1–6. <https://doi.org/10.1109/IECON48115.2021.9589102>
- Lee, S., Baek, Y.-M., Hyun, S., Bae, D.-H., 2021. Generation of adaptation strategies for dynamic reconfiguration of a system of systems. In: 16th International Conference of System of Systems Engineering, pp. 144–149. <https://doi.org/10.1109/SOSE52739.2021.9497463>
- Li, J., Manzano, W., Yamauchi, T., Matsuyama, N., Nakagawa, E.Y., Tei, K., 2024. Employing discrete controller synthesis for developing systems-of-systems controllers. In: 12th ACM/IEEE International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems, pp. 1–8. <https://doi.org/10.1145/3643655.3643875>
- Lopes, F., Loss, S., Mendes, A., Batista, T., Lea, R., 2016. SoS-Centric middleware services for interoperability in smart cities systems. In: 2nd International Workshop on Smart, pp. 1–6. <https://doi.org/10.1145/3009912.3009917>
- Loss, S., Colao, D., Cacho, N., Lopes, F., 2023. System of system strategy for multi-level interoperability for smart cities. In: IEEE International Smart Cities Conference, pp. 1–7. <https://doi.org/10.1109/ISC257844.2023.10293338>
- Maciel, M., Maia, P.H., Oliveira, F.C., Maciel, F., 2019. Adore: an adaptation-oriented requirement modeling approach for systems of systems. In: XXXIII Brazilian Symposium on Software Engineering, pp. 166–171. <https://doi.org/10.1145/3350768.3353814>
- Maciel, R. S.P., Valle, P. H.D., Santos, K.S., Nakagawa, E.Y., 2024. Systems interoperability types: a tertiary study. *ACM Comput. Surv.* 56 (10), 1–37.
- Maia, P.H., Vieira, L., Chagas, M., Yu, Y., Zisman, A., Nuseibeh, B., 2019. Cautious adaptation of defiant components. In: 34th IEEE/ACM International Conference on Automated Software Engineering, pp. 974–985. <https://doi.org/10.1109/ASE.2019.00094>
- Margarita, T., Chaudhary, H. A.A., Guevara, I., Ryan, S., Schieweck, A., 2021. The interoperability challenge: building a model-driven digital thread platform for CPS. In: International Symposium on Leveraging Applications of Formal Methods, pp. 393–413. https://doi.org/10.1007/978-3-030-89159-6_25
- Mittal, S., Kasdaglis, N., Harrell, L., Wittman, R.L., Gibson, J., Rocca, D., 2020. Autonomous and composable m&s system of systems with the simulation, experimentation, analytics and testing (SEAT) framework. In: Winter Simulation Conference, pp. 2305–2316. <https://doi.org/10.1109/WSC48552.2020.9384040>
- Mittal, S., Wittman, R.L., Gibson, J., Huffman, J., Miller, H., 2023. Providing a user extensible service-enabled multi-fidelity hybrid cloud-deployable SoS test and evaluation (T&E) infrastructure: application of modeling and simulation (M&S) as a service (MSaaS). *Information* 14 (10), 528. <https://doi.org/10.3390/INFO14100528>
- Nadira, B., Bouanaka, C., Bendjaballah, M., Djarri, A., 2020. Towards an UML-based SoS analysis and design process. In: 2020 International Conference on Advanced Aspects of Software Engineering, pp. 1–8. <https://doi.org/10.1109/FICLOUD57274.2022.00036>
- Naudet, Y., Panetto, H., Yilma, B.A., 2023. Towards cognitive interoperability in cyber-physical enterprises. *IFAC-PapersOnLine* 56 (2), 695–706.
- Neema, H., Szitapanovits, J., Steinbrink, C., Raub, T., Cornelien, B., Lehnhoff, S., 2019. Simulation integration platforms for cyber-physical systems. In: Workshop on Design Automation for CPS and IoT, pp. 10–19. <https://doi.org/10.1145/3313151.3313169>
- Neto, V. G.G., 2017. A model-based approach towards the building of trustworthy software-intensive systems-of-systems. In: 39th International Conference on Software Engineering Companion, pp. 425–428. <https://doi.org/10.1109/ICSE-C.2017.28>
- Neto, V. G.G., Paes, C.E., Rohling, A.J., Manzano, W., Nakagawa, E.Y., 2019. Modeling & simulation of software architectures of systems-of-systems: an industrial report on the brazilian space system. In: 2019 Spring Simulation Conference, pp. 1–12. <https://doi.org/10.23919/SPRINGSIM.2019.8732870>
- Nevelsteen, K., Wehlou, M., 2021. IPSME-idempotent publish/subscribe messaging environment. In: International Workshop on Immersive Mixed and Virtual Environment Systems, pp. 30–36. <https://doi.org/10.1145/3458307.3460966>
- Nielsen, C.B., Larsen, P.G., Fitzgerald, J., Woodcock, J., Peleska, J., 2015. Systems of systems engineering: basic concepts, model-based techniques, and research directions. *ACM Comput. Surv.* 48 (2), 1–41. <https://doi.org/10.1145/2794381>
- Nikolopoulos, B., Dimopoulos, A.C., Nikolaidou, M., Dimitrakopoulos, G., Anagnostopoulos, D., 2019. A system of systems architecture for the internet of things exploiting autonomous components. *Intern. J. Syst. Syst. Eng.* 9 (2), 167–199. <https://doi.org/10.1504/IJSSE.2019.100339>
- Nilsson, J., Javed, S., Albertsson, K., Delsing, J., Liwicki, M., Sandin, F., 2024. AI Concepts for system of systems dynamic interoperability. *Sensors* 24 (9), 2921. <https://doi.org/10.3390/S24092921>
- Nundloll, V., Elkhatib, Y., Elhabbash, A., Blair, G.S., 2020. An ontological framework for opportunistic composition of iot systems. In: IEEE International Conference on Informatics, IoT, and Enabling Technologies, pp. 614–621. <https://doi.org/10.1109/ICIOT48696.2020.9089467>
- Nweye, K., Kaspar, K., Buscemi, G., Fonseca, T., Pinto, G., Ghose, D., Duddukuru, S., Pratapa, P., Li, H., Mohammadi, J., Ferreira, L.L., Hong, T., Ouf, M., Capozzoli, A., Nagy,

- Z., 2025. Citylearn v2: energy-flexible, resilient, occupant-centric, and carbon-aware management of grid-interactive communities. *J. Build. Perform. Simul.* 18 (1), 17–38. <https://doi.org/10.1080/19401493.2024.2418813>
- Olsson, T., Axelsson, J., 2023. Systems-of-systems and digital twins: a survey and analysis of the current knowledge. In: 18th Annual System of Systems Engineering Conference, pp. 1–6. <https://doi.org/10.1109/SOSE59841.2023.10178527>
- Oquendo, F., 2019a. Architecting exogenous software-intensive systems-of-systems on the internet-of-vehicles with SoSADL. *Systems Engineering* 22 (6), 502–518. <https://doi.org/10.1002/SYS.21521>
- Oquendo, F., 2019b. Coping with uncertainty in systems-of-systems architecture modeling on the IoT with SoSADL. In: 14th Annual Conference System of Systems Engineering, pp. 131–136. <https://doi.org/10.1109/SYSESE.2019.8753842>
- Oquendo, F., 2019c. Dealing with uncertainty in software architecture on the internet-of-things with digital twins. In: International Conference on Computational Science and Its Applications, pp. 770–786. https://doi.org/10.1007/978-3-030-24289-3_57
- Oquendo, F., 2020. Fuzzy architecture description for handling uncertainty in IoT systems-of-systems. In: IEEE 15th International Conference of System of Systems Engineering, pp. 555–562. <https://doi.org/10.1109/SOSE50414.2020.9130553>
- Palumbo, F., Fanni, T., Sau, C., Rodríguez, A., Madroñal, D., Desnos, K., Morvan, A., Pelcat, M., Rubattu, C., Lazcano, R., et al., 2019. Hardware/software self-adaptation in CPS: the CERBERO project approach. In: *Embedded Computer Systems: Architectures, Modeling, and Simulation*, pp. 416–428. https://doi.org/10.1007/978-3-030-27562-4_30
- Parri, J., Patará, F., Sampietro, S., Vicario, E., 2019. JARVIS, A hardware/software framework for resilient industry 4.0 systems. In: *Software Engineering for Resilient Systems*, pp. 85–93. https://doi.org/10.1007/978-3-030-30856-8_6
- Perišić, A., Perišić, I., Perišić, B., 2023. Simulation-based engineering of heterogeneous collaborative systems—a novel conceptual framework. *Sustainability* 15 (11), 8804. <https://doi.org/10.3390/su15118804>
- Pescatore, M., Beery, P., 2024. Interoperability analysis via agent-based simulation. *J. Defense Model. Simul.* 21 (1), 103–116. <https://doi.org/10.1177/15485129221111171>
- Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M., 2008. Systematic mapping studies in software engineering. In: 12th International Conference on Evaluation and Assessment in Software Engineering, pp. 68–77. <https://doi.org/https://dl.acm.org/doi/10.5555/2227115.2227123>
- Petitdemange, F., Borne, I., Buisson, J., 2021. Design process for system of systems reconstructions. *Syst. Eng.* 24 (2), 69–82. <https://doi.org/10.1002/SYS.21567>
- Qasim, L., Hein, A.M., Oлару, S., Garnier, J.-L., Jankovic, M., 2023. System reconfiguration ontology to support model-based systems engineering: approach linking design and operations. *Syst. Eng.* 26 (4), 347–364. <https://doi.org/10.1002/SYS.21661>
- Raz, A.K., Bhuyian, M., Bricio-Neto, J.L., Santos, C., Maxwell, D., 2024. Conceptual, mathematical, and analytical foundations for mission engineering and system of systems analysis. *IEEE Syst. J.* 18 (3), 1549–1559. <https://doi.org/10.1109/JSYST.2024.3409231>
- Rehman, K., Kipouridis, O., Karnouskos, S., Frendo, O., Dickel, H., Lipps, J., Verzano, N., 2019. A cloud-based development environment using HLA and kubernetes for the co-simulation of a corporate electric vehicle fleet. In: 2019 IEEE/SICE International Symposium on System Integration, pp. 47–54. <https://doi.org/10.1109/SII.2019.8700423>
- Replication-Package, 2025. SLR Dataset. <https://github.com/ashfaq92/SoSLR>. Accessed: 2025-10-27.
- Rock, C.T., Soule, N., Toll, B., Do, E., Milligan, J.R., Paulini, M.S., 2019. Efficiently composing validated systems integration gateways for dynamic, diverse data. In: 2019 IEEE Military Communications Conference, pp. 46–53. <https://doi.org/10.1109/MILCOM47813.2019.9020714>
- Sadik, A.R., Ashfaq, M., Mäkitalo, N., Mikkonen, T., 2025. Urban air mobility as a system of systems: an LLM-enhanced holonic approach. In: 18th Annual System of Systems Engineering Conference, pp. 1–7. <https://doi.org/10.1109/SOSE66311.2025.11083807>
- Sadik, A.R., Bolder, B., Subasic, P., 2023. A self-adaptive system of systems architecture to enable its ad-hoc scalability: unmanned vehicle fleet-mission control center case study. In: 7th International Conference on Intelligent Systems, Metaheuristics & Swarm Intelligence, pp. 111–118. <https://doi.org/10.1145/3596947.3596949>
- Santos, D.S., Oliveira, B. R.N., Kazman, R., Nakagawa, E.Y., 2022. Evaluation of systems-of-systems software architectures: state of the art and future perspectives. *ACM Comput. Surv.* 55 (4), 1–35. <https://doi.org/10.1145/3519020>
- Schütz, J., Marx Gómez, J., 2022. A brief introduction into (de-)coupling lifecycles in net-centric systems-of-systems. In: International Conference on Technological Advancement in Embedded and Mobile Systems, pp. 251–261. https://doi.org/10.1007/978-3-031-56603-5_22
- Seghiri, A., Belala, F., Hameurlain, N., 2022. Modeling the dynamic reconfiguration in smart crisis response systems. In: 17th International Conference on Evaluation of Novel Approaches to Software Engineering, pp. 162–173. <https://doi.org/10.5220/0011069300003176>
- Shin, Y.-J., Liu, L., Hyun, S., Bae, D.-H., 2021. Platooning LEGOs: an open physical exemplar for engineering self-adaptive cyber-physical systems-of-systems. In: 2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 231–237. <https://doi.org/10.1109/SEAMS51251.2021.00038>
- Silva, E., Batista, T., Oquendo, F., 2020. On the verification of mission-related properties in software-intensive systems-of-systems architectural design. *Sci. Comput. Program.* 192, 102425. <https://doi.org/10.1016/J.SCICO.2020.102425>
- Song, J., Bae, D.-H., 2023. Continuous verification with acknowledged MAPE-K pattern and time logic-based slicing: a platooning system of systems case study. *Journal of Systems and Software* 206, 111840. <https://doi.org/10.1016/j.jss.2023.111840>
- Song, J., Kang, J., Hyun, S., Jee, E., Bae, D.-H., 2022. Continuous verification of system of systems with collaborative MAPE-K pattern and probability model slicing. *Inf. Softw. Technol.* 147, 106904. <https://doi.org/10.1016/J.INFSOF.2022.106904>
- Stadie, M., Rodemann, T., Burger, A., Jomrich, F., Limmer, S., Rebhan, S., Saeki, H., 2021. V2b vehicle to building charging manager. In: 5th International Electric Vehicle Technology Conference, pp. 1–6. <https://www.honda-ri.de/pubs/pdf/4658.pdf>
- Sudeikat, J., Köhler-Buße, M., 2022. On combining domain modeling and organizational modeling for developing adaptive cyber-physical systems. In: 14th International Conference on Agents and Artificial Intelligence - Volume 1, pp. 330–336. <https://doi.org/10.5220/0010881200003116>
- Sun, Q., Li, H., Wang, Y., Zhang, Y., 2022. Multi-swarm-based cooperative reconfiguration model for resilient unmanned weapon system-of-systems. *Reliab. Eng. Syst. Safety* 222, 108426. <https://doi.org/10.1016/J.RESS.2022.108426>
- Sun, Q., Li, H., Zhong, Y., Ren, K., Zhang, Y., 2024. Deep reinforcement learning-based resilience enhancement strategy of unmanned weapon system-of-systems under inevitable interferences. *Reliab. Eng. Syst. Safety* 242, 109749. <https://doi.org/10.1016/J.RESS.2023.109749>
- Sun, Y., Zhang, T., 2023. Research on autonomous reconstruction method for dependent combat networks. *IEEE Syst. J.* 17 (4), 6104–6113. <https://doi.org/10.1109/JSYST.2023.3282016>
- Teixeira, P.G., Lebtog, B. G.A., Dos Santos, R.P., Fernandes, J., Mohsin, A., Kassab, M., Neto, V. V.G., 2020. Constituent system design: a software architecture approach. In: IEEE International Conference on Software Architecture Companion, pp. 218–225. <https://doi.org/10.1109/ICSA-C50368.2020.00045>
- Thomas, C., Mirzaei, E., Wudka, B., Siefke, L., Sommer, V., 2021. Service-oriented reconfiguration in systems of systems assured by dynamic modular safety cases. In: European Dependable Computing Conference, pp. 12–29. https://doi.org/10.1007/978-3-030-86507-8_2
- Tolk, A., Barry, P., Doskey, S.C., 2022. Using modeling and simulation and artificial intelligence to improve complex adaptive systems engineering. *Intern. J. Model. Simul. Sci. Comput.* 13 (02), 2241004. <https://doi.org/10.1142/S1793962322410045>
- Wang, Z., Elkhatab, Y., Elhabbash, A., 2022. HolonCraft—an architecture for dynamic construction of smart home workflows. In: 9th International Conference on Future Internet of Things and Cloud, pp. 213–220. <https://doi.org/10.1109/FICLOUD57274.2022.00036>
- Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, pp. 1–10. <https://doi.org/10.1145/2601248.2601268>
- Wudka, B., Thomas, C., Siefke, L., Sommer, V., 2020. A reconfiguration approach for open adaptive systems-of-systems. In: IEEE International Symposium on Software Reliability Engineering Workshops, pp. 219–222. <https://doi.org/10.1109/ISSREW51248.2020.00076>
- Zabasta, A., Kunicina, N., Kondratjevs, K., Ribickis, L., 2020. Adaptive workflow of service oriented IoT architectures for small and distributed automation systems. In: 3rd International Conference on Applications of Intelligent Systems, pp. 1–6. <https://doi.org/10.1145/3378184.3378189>
- Zhang, H., Babar, M.A., Bai, X., Li, J., Huang, L., 2011. An empirical assessment of a systematic search process for systematic reviews. In: 15th Annual Conference on Evaluation & Assessment in Software Engineering, pp. 56–65. <https://doi.org/10.1049/IC.2011.0007>
- Zhang, Z., Elkhatab, Y., Elhabbash, A., 2023. NLP-Based generation of ontological system descriptions for composition of smart home devices. In: 2023 IEEE International Conference on Web Services, pp. 360–370. <https://doi.org/10.1109/ICWS60048.2023.00055>
- Zyrianoff, I., Gigli, L., Montori, F., Kamiński, C., Di Felice, M., 2021. Two-way integration of service-oriented systems-of-systems with the web of things. In: 47th Annual Conference of the IEEE Industrial Electronics Society, pp. 1–6. <https://doi.org/10.1109/IECON48115.2021.9589619>